THE COMPLETE HANDBOOK

OPERATING SYSTEMS MASTERY

From Fundamentals to Advanced Systems, Linux & Interviews

Beginner · Intermediate · Advanced · Interview Mastery

Processes · Threads · Scheduling · Synchronization · Deadlocks
Memory · Virtual Memory · File Systems · I/O · IPC · Storage
Networking · Linux Internals · Virtualization · Containers
100+ Interview Questions · Solved Numericals · 16 Projects

A SELF-CONTAINED LEARNING COURSE IN BOOK FORM

How to Use This Book

This is not a summary and not a list of topics. It is a complete, self-contained course that assumes you know almost nothing about operating systems and takes you all the way to the point where you can explain OS internals, solve numerical problems, write systems code, and answer senior-level interview questions with confidence.

You do not need another textbook alongside this one. Every concept is built from intuition first, then formalized with correct technical detail, then connected to real Linux behavior, real code, and the questions you will actually be asked. Read it front to back the first time — the order is deliberate, and later parts assume the earlier ones.

The Learning Path

Everything is organized along a single rising path. You will feel each part get a little deeper than the last, but never jump a step.



How Each Concept Is Taught

Important concepts are taught with the same repeatable structure so your brain learns how to learn them. You will not see all twelve labels spelled out every time — that would be noise — but every major idea moves through this arc:

Step	What it gives you
1. What is it?	A one-line, jargon-free definition.
2. Why do we need it?	The problem it solves — motivation before mechanism.
3. Simple intuition	An everyday analogy you already understand.
4. Technical explanation	The precise, correct definition with proper terms.
5. How it works internally	What the hardware and kernel actually do.
6. Example	A concrete, worked case — often in C or on the Linux shell.
7. Diagram	A picture that shows the real mechanism, not decoration.
8. Real-world example	Where this shows up in Linux, servers, or your laptop.
9. Related concepts	Links to ideas you have met or will meet.
10. Common mistakes	The traps that cost marks and interview offers.
11. Interview questions	How it is actually asked, with model answers.
12. Quick revision	The compressed version for the night before.

The Boxes — and What They Mean

Throughout the book, colored boxes flag the five things your eye should be able to find in seconds when revising:

> ▣ **KEY CONCEPT**
>
> The core idea, stated precisely. If you remember nothing else from a section, remember what is in these boxes.

> ⚑ **INTERVIEW TIP**
>
> Exactly how to phrase an answer, what the interviewer is really testing, and the follow-up they will ask next.

> ⚠ **COMMON MISTAKE**
>
> A misconception almost everyone holds at first. Catching these is what separates a memorized answer from an understood one.

> ⚑ **REAL WORLD**
>
> Where the concept lives in an actual system — Linux, a database, a web server, your own machine.

> ↻ **QUICK REVISION**
>
> The compressed, high-density recap. Read these to warm up, and again the night before an exam or interview.

Other Elements

Code blocks
Real, compilable C/POSIX and real Linux commands in monospace. Type them out — you learn systems by running things, not by reading about them.

Comparison tables
The exact "X vs Y" distinctions interviewers love. Part XX collects them all.

Solved numericals
Scheduling, paging, page replacement, Banker's, and disk problems worked step by step, with every calculation shown.

Check Your Understanding
End-of-part checkpoints: conceptual questions, application questions, interview questions, a practical experiment, and a clear "you have mastered this when…" bar.

> ⓘ **A NOTE ON DEPTH VS. SIMPLICITY**
>
> Where a simple explanation and a fully precise one differ, this book gives you the simple one first so you understand, then the precise one so you are correct. It never trades away technical accuracy for comfort — it just orders them so the accuracy actually lands.

The Fastest Way to Get Value

1. **First pass:** read a part fully, do its checkpoint, run its practical task.
2. **Second pass:** read only the **Key Concept** and **Quick Revision** boxes.
3. **Interview prep:** Part XVIII (question bank) + Part XX (comparison tables) + Part XXIII (final revision).
4. **Exam prep:** Part XIX (numericals) + each part's checkpoint + Part XXIII cheat sheet.

Let's begin.

Table of Contents

FRONT MATTER

How to Use This Book 2

FOUNDATIONS

Part I — OS Fundamentals & Architecture 7
Part II — Processes 20
Part III — Threads & Concurrency 31
Part IV — CPU Scheduling 39

COORDINATION & MEMORY

Part V — Process Synchronization 48
Part VI — Deadlocks 58
Part VII — Memory Management 65
Part VIII — Virtual Memory 72

SYSTEMS & DEVICES

Part IX — File Systems . . . . .
Part X — I/O Systems . . . . .
Part XI — Inter-Process Communication . . . . .
Part XII — Storage Systems . . . . .
Part XIII — Networking from an OS Perspective . . . . .

PRACTICAL & ADVANCED

Part XIV — Linux Practical OS . . . . .
Part XV — Advanced Operating Systems . . . . .
Part XVI — What Happens When... . . . . .
Part XVII — Practical Projects . . . . .

MASTERY & REVISION

Part XVIII — Interview Mastery (100+ Questions) . . . . .
Part XIX — Numerical Practice . . . . .
Part XX — Comparison Tables . . . . .
Part XXI — Visual Learning Index . . . . .
Part XXII — Mastery Checkpoints . . . . .
Part XXIII — Final Revision & Mastery Test . . . . .

PART I

OS Fundamentals & Architecture

Before processes, memory, or scheduling make sense, you need to know what an operating system is, why a computer is almost useless without one, and how a single program is kept from crashing the whole machine.

In this part: What an OS is · Resource manager & extended machine · Kernel and user space · User vs kernel mode · Privileged instructions · System calls · Traps, interrupts, exceptions · OS services · Protection & security · The boot process · Monolithic, micro, hybrid & modular kernels

PART I

OS Fundamentals & Architecture

A modern computer is a pile of extraordinarily fast but extraordinarily literal hardware. The operating system is the layer of software that turns that raw hardware into something a human — and a program — can actually use safely. This part builds the mental model that every later part depends on.

1.1 What Is an Operating System?

What is it? An **operating system (OS)** is the master program that sits between your applications and the physical hardware. It controls the hardware and provides services that every program can use, so that programs never have to talk to the raw metal directly.

Why do we need it? Imagine writing a program that plays a sound. Without an OS you would have to know the exact model of sound card, write the precise byte sequences it expects, manage the electrical timing, and make sure no other program touches the card at the same time. Now do that again for every keyboard, disk, network card, and GPU ever made. It is hopeless. The OS solves this once, for everyone, and hands you a simple request like "play these bytes."

> ▣ **KEY CONCEPT**
>
> An operating system does two big jobs at once: it is a **resource manager** (it decides who gets the CPU, memory, and devices, and when) and an **extended machine / abstraction layer** (it hides ugly hardware details behind clean, uniform interfaces like files and processes).

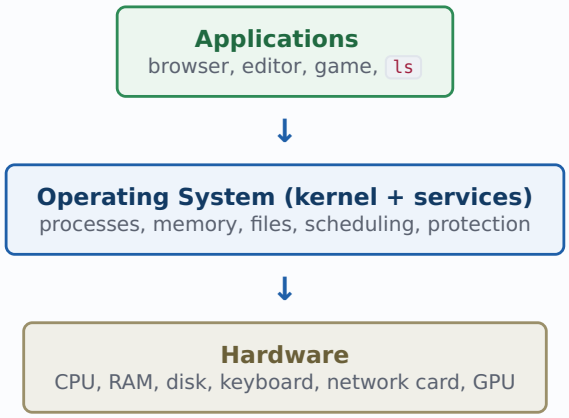
Simple intuition. Think of an OS as the manager of a busy shared workshop. Many workers (programs) want the same tools (CPU, memory, printer). The manager hands out tools fairly, stops two workers from grabbing the same tool at once, cleans up when a worker leaves, and gives everyone the same simple checkout process instead of making each worker learn how every tool is wired internally.

Technical explanation. The OS is a collection of software — centered on the **kernel** — that (1) manages hardware resources (processor time, memory, storage, I/O devices), (2) provides an execution environment for programs called the **process**, (3) exposes services through **system calls**, and (4) enforces **protection** so a fault or attack in one program cannot corrupt another or the OS itself.

How it works internally. The OS is loaded into memory at boot and stays resident. It runs in a privileged CPU mode so it can execute instructions ordinary programs cannot. When a program needs something only the OS can do (read a file, get more memory, send a packet), it makes a controlled request that switches the CPU into the

OS, the OS does the work, and control returns to the program. Between those requests, the OS is mostly idle — it is event-driven, woken by system calls and hardware interrupts.

Example. When you run `ls` in a terminal, the OS finds the program on disk, loads it into memory, gives it CPU time, lets it ask the file system for directory contents, sends its output to your terminal, and reclaims its memory when it exits — all without `ls` knowing anything about your specific SSD or display.



The OS is the mandatory middle layer: applications go through it to reach hardware.

Real-world example. Linux, Windows, macOS, Android, and iOS are operating systems. The same Chrome browser binary logic runs on wildly different laptops because the OS hides those differences. Your phone runs dozens of apps on one CPU because the OS shares it among them.

Related concepts. Kernel (§1.4), system calls (§1.7), processes (Part II), and protection (§1.10) are all facets of these two core jobs.

> ⚠ **COMMON MISTAKE**
>
> "The OS is the desktop, windows, and icons." That graphical shell is just one application running on top of the OS. You can run Linux with no graphics at all. The OS is the kernel and its services — the part that manages the hardware — not the wallpaper.

> ⚑ **INTERVIEW TIP**
>
> Asked "What is an operating system?", give the two-job answer in one breath: "It's the software that manages hardware resources and provides an abstraction layer so programs can use the hardware safely and portably through a uniform interface." Then be ready to expand either half.

1.2 Resource Manager & Extended Machine

These are the two classic lenses for understanding everything an OS does. Every feature in this book is one or the other (usually both).

The OS as a Resource Manager

The hardware has finite resources; many programs want them simultaneously. The OS multiplexes them — in **time** (the CPU runs process A for a few milliseconds, then B, then C, so fast it looks simultaneous) and in **space** (RAM is divided so each process gets its own region). It must do this fairly (no program starves), efficiently (little overhead), and safely (one program cannot read or wreck another's resources).

Resource	How the OS manages it	Covered in
CPU time	Scheduling: decides which process/thread runs next and for how long	Part IV
Main memory	Allocation, paging, virtual memory, protection	Parts VII–VIII
Storage	File systems, disk scheduling, free-space management	Parts IX, XII
I/O devices	Drivers, interrupts, DMA, buffering	Part X
Access / identity	Users, permissions, isolation	§1.10

The OS as an Extended (Virtual) Machine

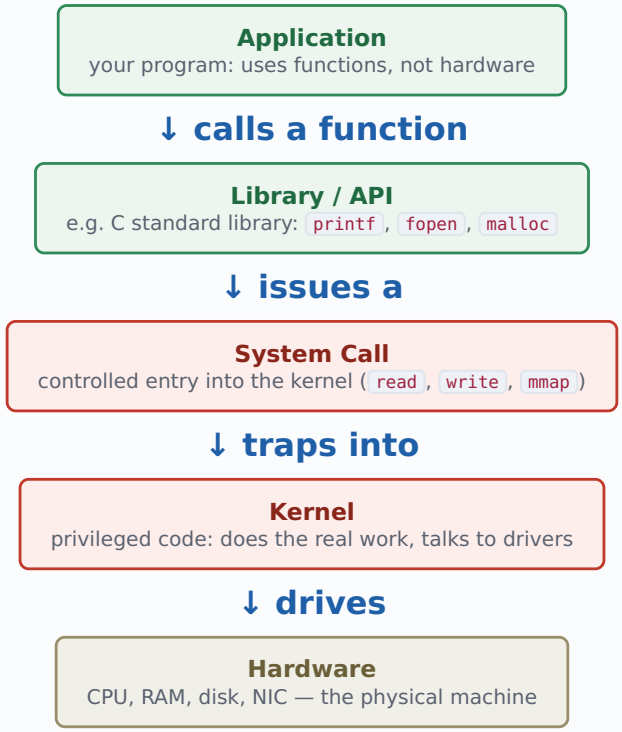
Raw hardware is hostile to program: disks store numbered blocks, not "files"; memory is one flat array shared by all; the CPU has no notion of "a program." The OS presents a friendlier **extended machine** where those hard things become easy abstractions:

Ugly hardware reality	Clean OS abstraction
Numbered disk sectors, no names	**Files** and directories with names
One shared physical RAM	A private **virtual address space** per program
One CPU, one instruction stream	Many concurrent **processes** and **threads**
Device-specific register pokes	Uniform `read()` / `write()` calls

> ▣ **KEY CONCEPT**
>
> **Abstraction** = hiding complexity behind a simpler interface. **Virtualization** = giving each program the illusion it has the whole machine to itself. The OS does both constantly: a file is an abstraction of disk blocks; a process is a virtualization of the CPU; virtual memory is a virtualization of RAM.

1.3 Hardware, OS, and Application: The Layered View

The single most useful picture in this book. Memorize it — nearly every question about "what happens when…" is really a question about how a request travels down and back up these layers.



The canonical stack. A request flows down; results (and interrupts) flow back up.

The two horizontal lines that matter most are: the **API boundary** (between your app and the library) and the **system-call boundary** (between user space and the kernel). The second is the famous "user/kernel line," and crossing it is not free — see §1.7.

1.4 Kernel, User Space, and Kernel Space

What is the kernel? The **kernel** is the core of the OS — the part that is always in memory, runs with full hardware privilege, and directly manages processes, memory, and devices. When people say "the OS did X," they usually mean the kernel did X.

Memory is conceptually split into two territories:

Kernel space
: The memory region where the kernel's code and data live. Only privileged (kernel-mode) code may access it. Protected from ordinary programs.

User space

Where ordinary programs (and their libraries) run. Each process gets its own user-space view. Code here cannot touch kernel space or other processes' memory directly.

> ▣ **KEY CONCEPT**
>
> **User space vs kernel space** is about memory and privilege territory. **User mode vs kernel mode** (next section) is about the CPU's current privilege state. They move together: code in user space runs in user mode; the kernel runs in kernel mode. Do not confuse the two axes — one is "where the memory is," the other is "what the CPU is allowed to do right now."

1.5 Dual-Mode Operation: User Mode and Kernel Mode

Why do we need it? If every program could run any instruction and touch any memory, one buggy or malicious program could halt the CPU, erase the disk, or read your passwords out of another program. To prevent that, the CPU itself supports (at least) two privilege levels.

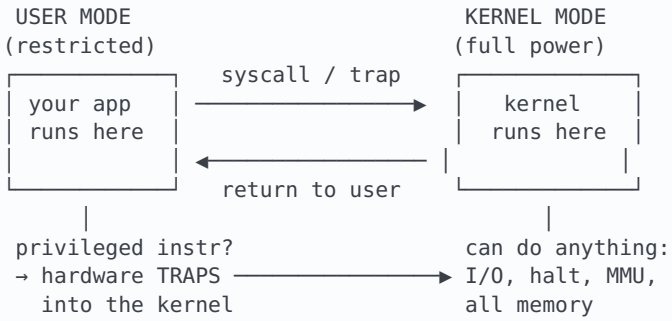
Kernel mode (supervisor / privileged mode)

The CPU can execute every instruction and access all memory and devices. The kernel runs here.

User mode

A restricted state. Privileged instructions are forbidden; memory access is limited to the process's own space. Applications run here.

How it works internally. A bit in a CPU register (the "mode bit," e.g. current privilege level / ring on x86) records the mode. In user mode, if a program attempts a privileged instruction or touches memory it does not own, the hardware immediately raises an **exception (trap)** and jumps into the kernel, which typically kills the offending program (the classic segmentation fault). The only legitimate ways to get from user mode into kernel mode are a **system call**, an **interrupt**, or an **exception** — all of which enter the kernel at fixed, trusted entry points. A program can never simply "switch itself" to kernel mode; that is the whole point.



Dual-mode operation. The mode bit gates what the CPU may do; controlled doorways cross it.

> ⚠ **COMMON MISTAKE**
>
> Thinking a program "calls the kernel like a normal function." It does not — a normal call cannot change privilege. A system call executes a special CPU instruction (e.g. `syscall` on x86-64) that atomically switches to kernel mode and jumps to a fixed kernel entry point. That hardware-enforced doorway is what makes the boundary safe.

1.6 Privileged Instructions

A **privileged instruction** is a machine instruction that can only be executed in kernel mode. These are exactly the operations that could compromise the whole system if any program could run them:

- Halting the CPU, disabling/enabling interrupts.
- Loading the page-table base register / switching address spaces (controlling the MMU).
- Direct access to I/O device registers.
- Modifying the mode bit or system control registers.
- Executing `hlt`, setting timers, flushing the TLB.

Attempting any of these in user mode raises a trap. This single mechanism — "some instructions are off-limits unless you are the kernel" — is the hardware foundation of all OS protection.

1.7 System Calls — and API vs System Call

What is it? A **system call** is the controlled mechanism by which a user-mode program requests a service from the kernel. It is the only door through which a program can ask the OS to do something privileged (open a file, allocate memory, create a process, send data on the network).

Simple intuition. You (a program) cannot walk into the bank vault (hardware). You slide a request form through a small, guarded window (the system call). A teller (the kernel) verifies your request, does it on your behalf, and slides the result back. You never touch the vault yourself.

How it works internally (x86-64 Linux, step by step):

1. The program puts the system-call number in a register (e.g. `rax`) and arguments in others (`rdi`, `rsi`, `rdx`…).
2. It executes the `syscall` instruction. The CPU switches to kernel mode and jumps to a fixed kernel entry point (the system-call handler).
3. The kernel looks up the number in the **system-call table**, validates the arguments, and runs the corresponding kernel function.

4. The result is placed in a register; the kernel executes a return-from-kernel instruction that switches back to user mode and resumes the program right after the `syscall`.

A RAW SYSTEM CALL VS. THE LIBRARY WRAPPER (LINUX, C)

```c
/* The C library gives you a friendly wrapper... */
ssize_t n = write(1, "hi\n", 3);   // fd 1 = stdout, 3 bytes

/* ...which internally does roughly this (conceptually): */
//   mov rax, 1        ; syscall number for write on x86-64
//   mov rdi, 1        ; arg1: file descriptor
//   mov rsi, buf      ; arg2: pointer to data
//   mov rdx, 3        ; arg3: byte count
//   syscall           ; <-- traps into the kernel
```

▣ KEY CONCEPT — API VS SYSTEM CALL

An **API** (Application Programming Interface) is the set of functions your code calls, such as the C library's `fopen()`, `printf()`, `malloc()`. A **system call** is the kernel entry underneath, such as `open`, `write`, `mmap`. Many API functions wrap one or more system calls — but not always: `strlen()` makes zero system calls, and one `printf()` may buffer many prints into a single `write`. The API is what you program against; the system call is what actually crosses into the kernel.

Aspect	API (library function)	System call
Lives in	User space (e.g. libc)	Kernel (entered via a trap)
Cost	Cheap — an ordinary function call	Expensive — mode switch + kernel work
Example	`printf`, `fopen`, `malloc`	`write`, `open`, `brk`/`mmap`
Portability	Portable across OSes (same C API)	OS-specific numbers & semantics
Purpose	Convenience, buffering, portability	The actual privileged service

Categories of system calls. Process control (`fork`, `execve`, `exit`, `wait`), file management (`open`, `read`, `write`, `close`), device management (`ioctl`), information (`getpid`, `time`), communication (`pipe`, `socket`, `shmget`), and protection (`chmod`, `setuid`).

⚑ INTERVIEW TIP

"Why are system calls expensive?" Because each one costs a **mode switch** (save user state, switch to kernel mode, run the handler, restore state) plus the kernel's own validation and work, and it can pollute CPU caches and pipelines. That is why libraries buffer — one big `write` beats a thousand tiny ones. This is also why high-performance systems batch I/O (`writev`, `io_uring`) to amortize the crossing.

> ⚑ **REAL WORLD**
>
> Run `strace ls` on Linux. You will see the actual stream of system calls `ls` makes — `openat`, `read`, `write`, `close`, `mmap`. This is the single best way to see the user/kernel boundary in action. We use it throughout Part XIV.

1.8 Traps, Interrupts, and Exceptions

All three are ways the CPU stops what it is doing and jumps into the kernel. Interviewers love the distinction because candidates blur it. Here is the clean version:

Term	Source	Timing	Meaning & example
Interrupt	External hardware (device)	Asynchronous	A device signals it needs attention, unrelated to the current instruction. E.g. keyboard press, packet arrival, disk finished, timer tick.
Exception / Fault	The running instruction (CPU)	Synchronous	The current instruction did something the CPU can't complete. E.g. divide-by-zero, invalid memory access, **page fault**.
Trap	A deliberate instruction	Synchronous	The program intentionally asks to enter the kernel. E.g. a **system call**, a debugger breakpoint. (A "software interrupt.")

> ▣ **KEY CONCEPT**
>
> **Asynchronous vs synchronous** is the crisp dividing line. An **interrupt** is asynchronous — it comes from the outside world and can arrive between any two instructions. An **exception** and a **trap** are synchronous — they are caused by the instruction currently executing. A trap is the intentional kind of exception ("I meant to do that"); a fault is the unintended kind ("something went wrong").

How the kernel responds. The CPU consults the **interrupt vector table** (a table of handler addresses indexed by interrupt/exception number), saves the current state, switches to kernel mode, and runs the matching **interrupt/exception handler**. When the handler finishes, the CPU restores state and resumes — either the interrupted program (for an interrupt) or, for a fault it fixed (like a page fault), re-runs the faulting instruction.

1.9 OS Services

Everything the OS offers falls into a handful of service categories. You have now met the mechanisms; these are the user-facing results.

- **Program execution** — load, run, and end programs (Part II).
- **I/O operations** — read/write files and devices without touching hardware (Part X).

- **File-system manipulation** — create, delete, search, and protect files (Part IX).
- **Communication** — let processes exchange data locally or over a network (Parts XI, XIII).
- **Error detection** — catch hardware and software faults and respond safely.
- **Resource allocation** — share CPU, memory, and devices among many programs (Parts IV, VII).
- **Accounting & logging** — track who used what.
- **Protection & security** — isolate programs and users (§1.10).

1.10 Protection and Security

These words are related but not identical, and mixing them up is a classic slip.

Protection

Controlling access by programs and users to the resources inside the system — e.g. stopping process A from reading process B's memory, or a user from deleting another user's files. It guards against **accidents and internal misuse**.

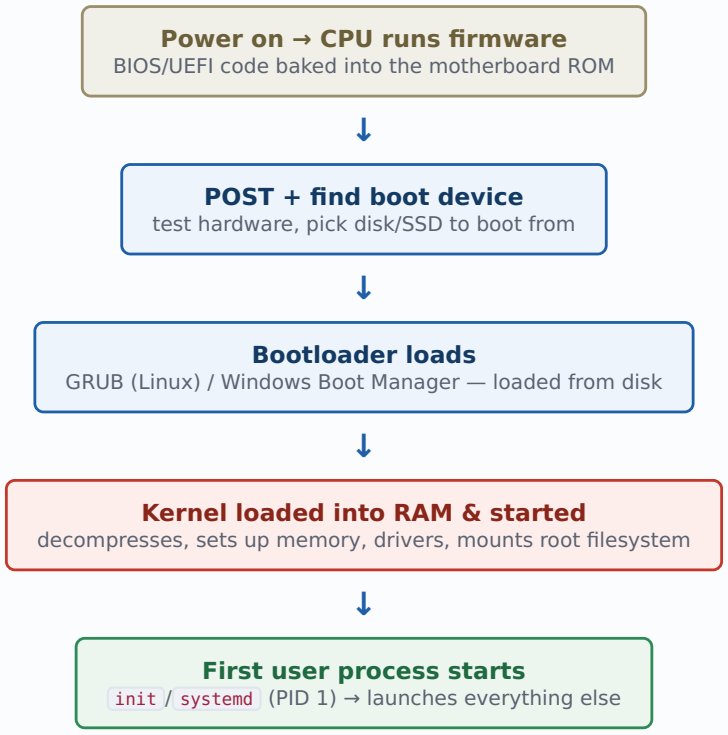
Security

Defending the system against external threats — malware, network attackers, stolen credentials. It guards against **deliberate, often external, attacks**.

Mechanisms include dual-mode operation, memory protection via the MMU (Part VII), user/group permissions (Part IX), and modern additions like ASLR and namespaces (Part XV). Protection is the foundation; security is built on top of it.

1.11 The Boot Process

The chicken-and-egg problem: the OS manages the hardware, but the OS is a program that must itself be loaded and run by the hardware. **Booting** (from "bootstrapping" — pulling yourself up by your bootstraps) is how the machine climbs from bare power-on to a running OS.



Cold boot to login. Each stage loads and hands control to the next, larger stage.

The detail-by-detail version — including what the firmware, bootloader, and PID 1 each do — is in Part XVI, "What happens when I turn on my computer?" For now, the shape is enough: firmware → bootloader → kernel → init → your login.

1.12 OS Architectures: Monolithic, Micro, Hybrid, Modular

Where should all this OS code live — one giant privileged blob, or many small pieces? This design choice defines an OS's **architecture**, and it is a favorite interview and exam topic.

Monolithic Kernel

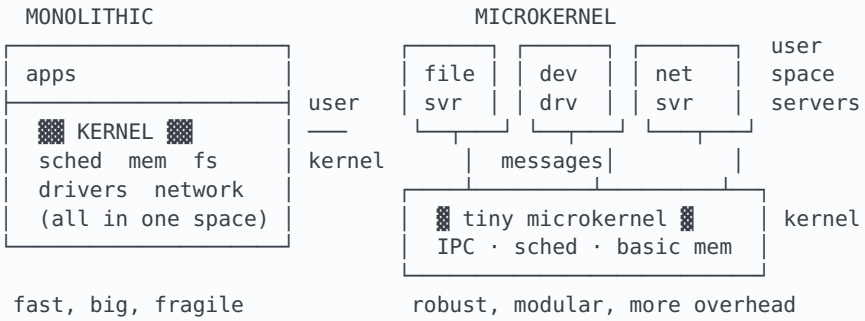
The entire OS — scheduling, memory, file systems, drivers, network stack — runs together in kernel space as one large program. Components call each other directly as fast function calls.

+ Very fast (no crossing between components). **−** Huge, complex; a bug in any driver can crash the whole kernel; hard to maintain in isolation. **Examples:** classic UNIX, Linux (though modular), BSD.

Microkernel

Keep the kernel tiny: only the bare essentials (scheduling, basic memory, and inter-process messaging) run in kernel mode. Everything else — file systems, drivers, networking — runs as ordinary **user-space servers** that talk via message passing.

+ Robust and secure (a crashed driver doesn't take down the kernel; just restart the server); modular; easier to verify. **−** Slower — lots of message passing and mode switches. **Examples:** Minix, QNX, seL4, the Mach core.



Monolithic puts everything in kernel space; microkernel pushes services out to user space.

Hybrid Kernel

A pragmatic middle ground: a mostly-monolithic kernel for speed, but structured with some microkernel ideas and message passing. **Examples:** Windows NT, macOS (XNU = Mach + BSD).

Modular Kernel

A monolithic core that can **load and unload components at runtime** as **modules** — you get monolithic speed with some of the flexibility of a microkernel. **Example:** Linux, via **loadable kernel modules** (a driver you `insmod` / `modprobe` without recompiling the kernel; see Part XV). Modern Linux is best described as a modular monolithic kernel.

Property	Monolithic	Microkernel	Hybrid	Modular
Where services run	Kernel space	User space	Mostly kernel	Kernel (loadable)
Speed	Highest	Lower	High	High
Fault isolation	Poor	Excellent	Medium	Poor–Medium
Extensibility	Hard	Easy	Medium	Easy (modules)
Example	Classic UNIX	QNX, seL4	Windows, macOS	Linux

⚠ **COMMON MISTAKE**

Saying "Linux is a pure monolithic kernel." It is monolithic in that services run in kernel space, but it is modular — drivers and features load and unload at runtime as kernel modules. The precise answer is "modular monolithic," and saying so signals real understanding.

✓ Check Your Understanding — Part I

CONCEPTUAL (EXPLAIN IN YOUR OWN WORDS)

1. State the two fundamental jobs of an operating system and give an example of each.
2. What is the difference between user space/kernel space and user mode/kernel mode?
3. Why can't a user program simply switch itself into kernel mode?
4. Distinguish an interrupt, an exception, and a trap by source and timing.
5. Explain the difference between an API and a system call.

APPLICATION

1. You call `printf("hi")` a thousand times in a loop. Explain why this may result in far fewer than a thousand `write` system calls.
2. A driver has a bug that dereferences a bad pointer. Predict the outcome on a monolithic kernel vs. a microkernel, and why.
3. Order these boot stages: init/systemd, bootloader, firmware (UEFI), kernel.

INTERVIEW-LEVEL

1. "Why are system calls expensive, and how do real systems reduce that cost?"
2. "Is Linux monolithic or a microkernel? Defend your answer."

CAN YOU EXPLAIN THIS? (NO NOTES)

The user/kernel boundary and the three ways to cross it · API vs system call · resource manager vs extended machine · monolithic vs microkernel trade-offs.

⚑ **PRACTICAL TASK**

On any Linux machine (or WSL), run `strace -c ls`. Read the summary: which system calls did `ls` make, and how many times? Identify one call that reads directory data and one that writes your terminal. You have just watched the abstraction stack work.

Mastered when... you can draw the Application → Library → System Call → Kernel → Hardware stack from memory, explain what enforces each boundary, and describe exactly what happens (in hardware and software) during a single `write()` call.

PART II

Processes

A program is a dead file on disk. A process is that program brought to life — given memory, a CPU, and an identity. Understanding the process is understanding the single most important object the operating system manages.

In this part: Program vs process · The process address space · The PCB · Five process states & transitions · `fork()`, `exec()`, `wait()` · Parent/child trees · Zombies, orphans & daemons · Context switching · Scheduling queues · CPU vs I/O bursts · Process isolation

PART II

Processes

Everything a computer does for you happens inside a process. When you open a browser, compile code, or run a server, the OS creates processes, feeds them CPU time, gives each its own protected memory, and cleans up after them. Master the process and half of operating systems falls into place.

2.1 Program vs Process

What is it? A **program** is a passive, static file on disk — a sequence of instructions and data (e.g. `/usr/bin/ls`). A **process** is a program in execution: an active entity with its own memory, CPU registers, a program counter, open files, and a unique identity.

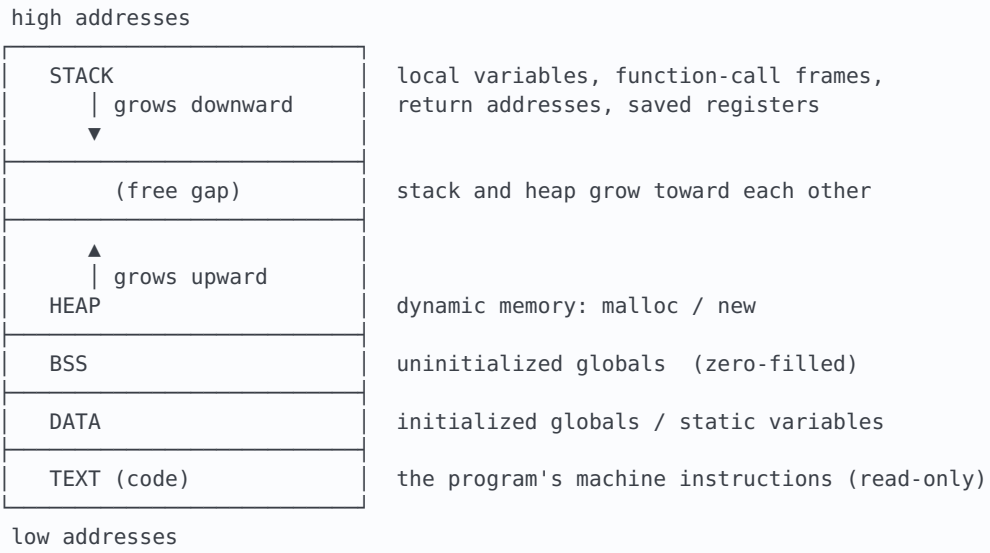
Simple intuition. A program is a recipe in a cookbook. A process is you actually cooking it — using a kitchen (memory), your hands (CPU), and a bookmark showing which step you are on (program counter). The same recipe can be cooked by two people at once in two kitchens: one program, two independent processes.

> ▣ **KEY CONCEPT**
>
> **Program : Process = Recipe : Cooking.** One program can spawn many processes (open your terminal twice — two `bash` processes from one `bash` program). A process is a program plus its execution state: memory image, registers, PC, kernel bookkeeping.

Aspect	Program	Process
Nature	Passive / static	Active / dynamic
Location	On disk (a file)	In memory (RAM), executing
Lifetime	Exists until deleted	Exists from creation to termination
Resources	None (just bytes)	CPU time, memory, open files, PID
Count	One file	Many processes from the same program

2.2 The Process Address Space

When the OS launches a process, it gives it a private **virtual address space** — a fresh, protected view of memory the process believes is entirely its own. That space is organized into segments. Knowing this layout explains stack overflows, `malloc`, and countless bugs.



A process's virtual address space. Text/data come from the program file; heap and stack grow at run time.

Text

The machine code. Read-only (so it can be shared between processes running the same program) and protected from accidental writes.

Data / BSS

Global and static variables. Data holds initialized ones; BSS holds zero-initialized ones (stored compactly as "reserve N zero bytes").

Heap

Dynamically allocated memory (`malloc` / `new`). Grows upward as you allocate; managed by the allocator on top of the `brk` / `mmap` syscalls.

Stack

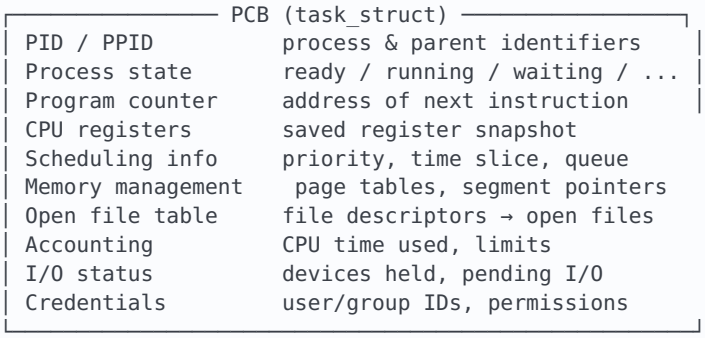
Function call frames: local variables, arguments, return addresses. Grows downward; automatically freed as functions return.

> ⚠ **COMMON MISTAKE**
>
> Confusing **stack** and **heap**. Locals live on the stack and vanish when the function returns — returning a pointer to a local is a bug. `malloc`'d memory lives on the heap and survives until you `free` it — forgetting to free is a memory leak.

2.3 The Process Control Block (PCB)

What is it? The **Process Control Block** is the kernel's data structure that stores everything the OS needs to know about a process. One PCB exists per process; it is the process, as far as the kernel is concerned. (In Linux the PCB is `struct task_struct`.)

Why do we need it? The CPU can run only one process at a time per core. To pause process A and later resume it exactly where it left off, the OS must save A's complete state somewhere — that somewhere is the PCB.



The PCB: the kernel's complete record of one process. Saving/restoring it is the essence of a context switch.

> ▣ **KEY CONCEPT**
>
> The PCB is the anchor for three big ideas at once: **context switching** (save/restore its registers & PC), **scheduling** (its state and priority determine what runs next), and **isolation** (its memory-management fields point to a private address space). The kernel keeps all PCBs in lists/queues so it can find and manage every process.

2.4 Process States and Transitions

A process is not always running — most of the time it is waiting for something (the CPU, or an I/O device). The OS models this with a small set of **states**. The five-state model is the one to know cold.

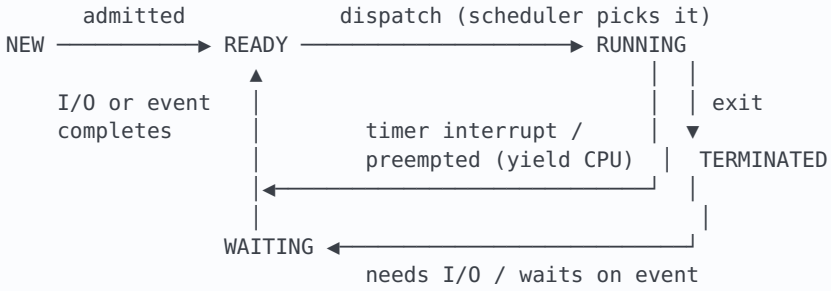
New
: The process is being created (PCB allocated, memory being set up).

Ready
: Loaded and willing to run — just waiting for the CPU. Sits in the **ready queue**.

Running
: Currently executing on a CPU. Only one process per core is here at any instant.

Waiting / Blocked
: Cannot proceed until some event happens (disk read completes, a lock is released, input arrives). Not competing for the CPU.

Terminated
: Finished executing; the OS is reclaiming its resources.



The five-state process model. Transitions are driven by the scheduler, timer interrupts, and I/O events.

The key transitions:

- **Ready → Running** (dispatch): the scheduler selects the process and gives it the CPU.
- **Running → Ready** (preemption): its time slice expired, or a higher-priority process arrived. It is still runnable — just paused.
- **Running → Waiting** (block): it requested I/O or a resource that is not ready. It voluntarily gives up the CPU.
- **Waiting → Ready** (wakeup): the awaited event happened; it rejoins the ready queue (note: it does not go straight to Running).
- **Running → Terminated**: it called `exit()` or was killed.

> ⚠ **COMMON MISTAKE**
>
> Thinking a process goes Waiting → Running directly when its I/O finishes. It does not — it goes Waiting → **Ready**, then must be scheduled before it runs. The CPU might be busy with someone else. This distinction matters in scheduling questions.

> ⚑ **INTERVIEW TIP**
>
> Draw the five-state diagram and name every transition and its cause. Then mention that real kernels add states: Linux has `TASK_RUNNING` (covers ready+running), `TASK_INTERRUPTIBLE` / `UNINTERRUPTIBLE` (two flavors of waiting), and `TASK_ZOMBIE`. Naming real kernel states signals depth.

2.5 Process Creation with fork()

What is it? On UNIX/Linux, new processes are born by **cloning**. The `fork()` system call creates a near-exact copy of the calling process. The original is the **parent**; the copy is the **child**. This builds a **process tree** rooted at PID 1 (`init` / `systemd`).

The famous trick — one call, two returns. `fork()` is called once but returns twice: once in the parent (returning the child's PID) and once in the child (returning 0). That return value is how each copy knows who it is.

FORK() — ONE CALL, TWO PROCESSES

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();              // clone this process here

    if (pid < 0) {
        perror("fork failed");   // no child was created
    } else if (pid == 0) {
        printf("Child : my PID=%d, parent=%d\n", getpid(), getppid());
    } else {
        printf("Parent: my PID=%d, child=%d\n", getpid(), pid);
        wait(NULL);                 // wait for the child to finish
    }
    return 0;
}
```

How it works internally. The kernel allocates a new PCB, copies the parent's state (open files, credentials, and a logical copy of its address space), assigns a new PID, and marks the child Ready. Copying all of memory would be wasteful, so modern kernels use **copy-on-write (COW)**: parent and child share the same physical pages, marked read-only; only when one writes does the kernel duplicate that page. This makes `fork()` cheap. (COW returns in Part VIII.)

⚠ **COMMON MISTAKE**

Assuming the parent runs before the child (or vice versa). After `fork()`, both are Ready and the scheduler decides the order — it is nondeterministic. Never write code that depends on which runs first; use `wait()` or synchronization if order matters.

2.6 Running a New Program with exec()

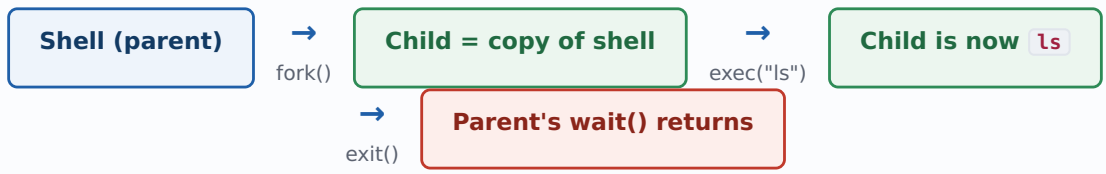
`fork()` gives you a copy of the same program. To run a different program, the child calls a member of the `exec()` family (`execl`, `execvp`, `execve`…). `exec()` **replaces** the current process's memory image with a new program — same PID, brand-new text/data/heap/stack. It does not return on success (there is nothing to return to; the old program is gone).

▣ **KEY CONCEPT — THE FORK/EXEC PATTERN**

This is how every shell launches every command: **fork** to create a child, then **exec** in the child to become the desired program, while the parent **wait**s. Separating "create a process" (`fork`) from "run a program" (`exec`) is a deliberate, powerful UNIX design — between the two, the child can redirect I/O, close files, or set up pipes (Part XI).

FORK + EXEC + WAIT: HOW A SHELL RUNS "LS -L"

```c
pid_t pid = fork();
if (pid == 0) {                        // --- child ---
    execlp("ls", "ls", "-l", NULL);    // become 'ls'; never returns if OK
    perror("exec failed");             // only reached on error
    _exit(1);
} else {                               // --- parent (the shell) ---
    int status;
    waitpid(pid, &status, 0);          // block until child finishes
    printf("child exited with %d\n", WEXITSTATUS(status));
}
```



The universal fork → exec → wait lifecycle behind every shell command.

2.7 Termination and wait()

A process ends by calling `exit(status)` (voluntary) or by receiving a fatal signal like `SIGKILL` (involuntary). On termination the kernel closes the process's files, frees its memory, and keeps a small **exit status** around so the parent can learn how the child fared.

The parent collects that status with `wait()` / `waitpid()`. This is called **reaping**. Until the parent reaps it, the dead child lingers as a **zombie** (next section). `wait()` also lets a parent synchronize with a child — blocking until it finishes.

2.8 Zombies, Orphans, and Daemons

Three special process conditions that interviewers love and beginners confuse. Learn them as a set.

Zombie Process

What: a child that has terminated but whose parent has not yet `wait()`ed for it. It is dead but still occupies a PCB slot (holding its exit status). **Why it exists:** so the parent can still read the exit status after the child is gone. **Danger:** a parent that never reaps its children leaks PCB entries; enough zombies can exhaust the process table. **Fix:** the parent must `wait()`, or handle `SIGCHLD`.

CREATING (AND OBSERVING) A ZOMBIE

```
pid_t pid = fork();
if (pid == 0) {
    _exit(0);              // child dies immediately
} else {
    sleep(30);             // parent naps WITHOUT wait() -> child is a zombie
    // run 'ps aux | grep defunct' now: you'll see it as <defunct>
}
```

Orphan Process

What: a child whose parent terminated first, leaving the child still running. **Who adopts it:** the kernel **reparents** orphans to `init`/`systemd` (PID 1), which dutifully reaps them when they finish. So orphans are harmless — they always get a responsible foster parent.

Daemon Process

What: a long-running background service with no controlling terminal — web servers, `sshd`, `cron`. **How it is made:** a process `fork`s, the parent exits (so the child becomes an orphan adopted by PID 1), and the child calls `setsid()` to start a new session, detaching from any terminal. The trailing **d** in names like `sshd`, `httpd`, `systemd` means "daemon."

Type	State	Parent	Problem?
Zombie	Dead, not yet reaped	Alive but hasn't waited	Yes — leaks PCB slots
Orphan	Still running	Dead; adopted by PID 1	No — PID 1 reaps it
Daemon	Running (background)	PID 1 (intentionally)	No — by design

⚠ **COMMON MISTAKE**

"You kill a zombie with `kill -9`." You cannot — it is already dead; there is nothing to kill. A zombie disappears when its parent reaps it. If the parent won't, kill the parent; the zombie is then reparented to PID 1, which reaps it instantly.

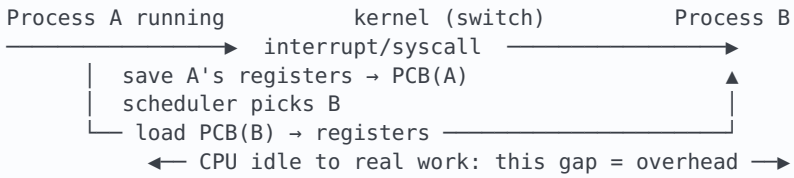
2.9 Context Switching

What is it? A **context switch** is the act of saving the state of the currently running process and restoring the state of another, so the CPU can switch from running process A to process B. It is the mechanism that makes one CPU appear to run many processes at once.

Simple intuition. A single receptionist (CPU) serving many customers (processes). To switch customers, the receptionist writes down exactly where they left off with customer A (save context), pulls out customer B's file (restore context), and continues. The note-taking itself takes time and serves no customer — that is **overhead**.

How it works internally. Triggered by a timer interrupt (preemption), a blocking system call, or a higher-priority wakeup, the kernel:

1. Saves the CPU registers and program counter of A into A's PCB.
2. Updates A's state (Running → Ready or Waiting) and picks B via the scheduler.
3. Loads B's registers, PC, and memory context (switches page tables / address space) from B's PCB.
4. Returns to user mode; B resumes exactly where it last stopped.



A context switch. The shaded gap does no useful application work — pure overhead.

> ⚑ **INTERVIEW TIP**
>
> "Why does context switching have overhead?" Two reasons: (1) the direct cost of saving/restoring registers and switching address spaces, and (2) the indirect cost — the new process starts with cold CPU caches and TLB (which may be flushed), so it runs slower until they warm up. This is why excessive switching (e.g. too many threads) hurts performance. Switching between threads of one process is cheaper: same address space, so no page-table switch or full TLB flush.

2.10 Scheduling Queues, CPU Bursts, and I/O Bursts

The OS keeps processes organized in queues as they move between states:

- **Job queue** — all processes in the system.
- **Ready queue** — processes in memory, Ready, waiting for the CPU.
- **Device / wait queues** — one per device; processes blocked on that device's I/O.

A process's life alternates between two kinds of activity:

CPU burst
: A stretch of computation using the CPU.

I/O burst
: A stretch of waiting for input/output (disk, network, user).

This **CPU–I/O burst cycle** is fundamental to scheduling (Part IV). **CPU-bound** processes (e.g. video encoding) have long CPU bursts; **I/O-bound** processes (e.g. a text editor) have short CPU bursts and lots of waiting. Good schedulers favor I/O-bound processes to keep devices busy.

2.11 Process Isolation

What & why. Each process gets its own virtual address space, so process A literally cannot see or corrupt process B's memory — a wild pointer in A crashes only A. This **isolation** is enforced by the hardware MMU (Part VII) under the kernel's control, and it is the reason one misbehaving app doesn't take down your whole machine.

Isolation is a double-edged sword: it keeps processes safe from each other, but it also means they cannot casually share data. To communicate, they need explicit, kernel-mediated mechanisms — which is exactly what **inter-process communication** provides.

2.12 Inter-Process Communication (Overview)

Because processes are isolated, the OS offers controlled channels for them to exchange data. There are two broad families:

- **Shared memory** — two processes map the same region of physical memory; fastest, but they must synchronize access themselves.
- **Message passing** — processes send/receive discrete messages via the kernel (pipes, message queues, sockets); safer and simpler, but involves copying and system calls.

The full toolkit — pipes, named pipes, message queues, shared memory, signals, sockets, and RPC — with working C examples is **Part XI**. For now, just hold the idea: isolation is the default; IPC is how you deliberately poke holes in it.

✓ Check Your Understanding — Part II

CONCEPTUAL

1. Give the recipe/cooking analogy for program vs process and state what a process has that a program lacks.
2. Name the four main segments of a process address space and what each holds.
3. List the five process states and the cause of every transition between them.
4. What is stored in a PCB, and why is it central to context switching?
5. Define zombie, orphan, and daemon in one sentence each.

APPLICATION

1. A `fork()`'d program prints a line before and after the fork. If the line before says "start" and the line after says "hi", how many total lines are printed and why?
2. Your server spawns worker children but never calls `wait()`. After a day it can't create new processes. Diagnose the bug and give two fixes.
3. Explain why switching between two threads of one process is cheaper than switching between two processes.

INTERVIEW-LEVEL

1. "Why does `fork()` return twice, and how does copy-on-write make it cheap?"
2. "Walk me through exactly what happens, step by step, when a shell runs `grep foo file`."

CAN YOU EXPLAIN THIS? (NO NOTES)

The five-state diagram with every transition · the fork/exec/wait pattern · what a context switch saves and restores (and why it has overhead) · zombie vs orphan.

⚑ PRACTICAL TASK

Compile the `fork()` example. Then write a version where the child `_exit`s and the parent `sleep`s for 20 s without `wait()`. While it sleeps, run `ps -el | grep defunct` in another terminal and find your zombie. Then add `wait(NULL)` and confirm the zombie is gone.

Mastered when... you can write a fork/exec/wait program from scratch, predict its output, explain the process tree it creates, and describe what the kernel does to its PCBs at each step.

PART III

Threads & Concurrency

A process gives you isolation. A thread gives you a way to do several things at once inside one process, sharing its memory. That sharing is the source of both a thread's power and every concurrency bug you will ever chase.

In this part: What a thread is · Process vs thread · What's shared vs private · Why threads · User-level vs kernel-level threads · Many-to-one, one-to-one, many-to-many · The TCB · Concurrency vs parallelism · Multi-core · Race conditions · Critical sections · Thread safety

PART III

Threads & Concurrency

If a process is a running program with its own memory, a thread is a single flow of execution within that memory. Threads let one program keep a UI responsive while it downloads a file, or serve thousands of users at once — but because they share memory, they force us to confront concurrency head-on.

3.1 What Is a Thread?

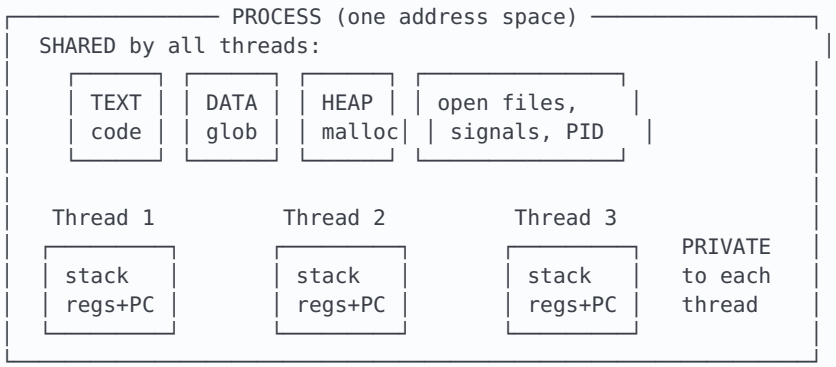
What is it? A **thread** (short for "thread of execution") is the smallest unit the CPU schedules — a single sequential flow of instructions with its own program counter, registers, and stack, running inside a process and sharing that process's memory and resources with its sibling threads.

Simple intuition. A process is a kitchen; a thread is a cook. One cook (single-threaded) does everything in sequence. Add more cooks (threads) in the same kitchen and they work simultaneously, sharing the same ingredients, counters, and fridge (memory). They get more done — but if two cooks grab the same knife at once, you have a problem (a race condition).

Why do we need it? Before threads, doing two things at once meant two processes, which are expensive to create and cannot easily share data. Threads are lightweight and share memory by default, so a single program can naturally overlap work — compute while waiting on I/O, or spread computation across CPU cores.

3.2 Process vs Thread — What's Shared, What's Private

This is the most-asked threading question. The answer hinges on one picture: threads of a process share almost everything, but each keeps its own execution state.



Threads share the process's code, globals, heap, and OS resources; each has its own stack, registers, and program counter.

SHARED across threads of a process	PRIVATE to each thread
Code (text segment)	Stack (local variables, call frames)
Global & static data	CPU registers
Heap (`malloc`'d memory)	Program counter (PC)
Open file descriptors, sockets	Thread ID (TID)
Process ID, signal handlers, working dir	Errno, thread-local storage (TLS)

▣ **KEY CONCEPT**

Threads share **memory** (so they can cooperate cheaply — no copying) but keep their own **stack + registers + PC** (so each can be at a different point in the code). Sharing memory is why threads are fast and why they need synchronization: two threads writing the same global with no lock is a data race.

Aspect	Process	Thread
Memory	Own private address space	Shares the process's address space
Creation cost	Heavy (new address space, COW)	Light (just a stack + PCB-like TCB)
Context switch	Expensive (switch page tables, flush TLB)	Cheaper (same address space)
Communication	Needs IPC (pipes, shared mem)	Shared variables (direct)
Isolation / safety	Strong — a crash stays contained	Weak — one thread's bad write can corrupt all
One crashes	Others unaffected	Can take down the whole process

⚠ **COMMON MISTAKE**

"Threads have their own heap." No — the **heap is shared**. If thread A `malloc`s and passes the pointer to thread B, B can use it. Only the **stack** is per-thread. This is precisely why passing a pointer to a local (stack) variable of one thread to another thread is dangerous.

3.3 Why Threads? (And When Not To)

- **Responsiveness** — one thread handles the UI while another does slow work; the app never freezes.
- **Resource sharing** — threads share memory for free; no IPC plumbing needed.
- **Economy** — creating and switching threads is far cheaper than processes.
- **Scalability / parallelism** — on a multi-core CPU, threads run truly simultaneously, one per core, cutting wall-clock time.

The cost: shared memory means shared bugs. Race conditions, deadlocks, and hard-to-reproduce "Heisenbugs" are the price. And more threads is not always faster — see the tip below.

> ⚑ **INTERVIEW TIP**
>
> "Why can too many threads reduce performance?" Because (1) each thread adds context-switching and scheduling overhead; (2) beyond the core count, threads compete for the same cores, so you pay switching cost without gaining parallelism; (3) more threads means more lock contention (they serialize on shared locks) and more cache thrashing. The sweet spot for CPU-bound work is roughly one thread per core; for I/O-bound work you can use more, since threads spend time blocked.

3.4 User-Level vs Kernel-Level Threads

Who manages the threads — a user-space library, or the kernel? This choice has real consequences.

User-level threads (ULT)
: Managed entirely by a library in user space; the kernel sees only one process and knows nothing of the threads. **+** Very fast to create/switch (no system call); portable. **−** If one thread makes a blocking system call, the whole process blocks (the kernel blocks the one process it can see); cannot use multiple cores.

Kernel-level threads (KLT)
: Managed by the kernel, which schedules each thread individually. **+** One thread blocking doesn't stop the others; real parallelism across cores. **−** Creation and switching involve the kernel, so they're slower than ULTs.

Modern systems (Linux, Windows) use kernel-level threads — every `pthread` maps to a kernel-scheduled thread — because true multi-core parallelism and non-blocking behavior matter more than the last drop of switch speed.

3.5 Multithreading Models

These describe how user threads map onto kernel threads.

Many-to-One

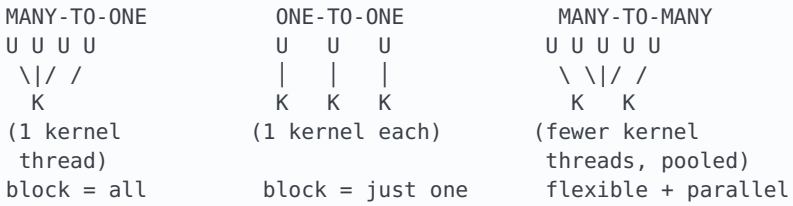
Many user threads map to a single kernel thread. Thread management is cheap and in user space, but one blocking call blocks everyone, and no multi-core parallelism. (Old "green threads.")

One-to-One

Each user thread maps to its own kernel thread. True concurrency and parallelism; a block affects only that thread. Cost: creating many threads creates many kernel threads. This is what **Linux and Windows use today**.

Many-to-Many

Many user threads multiplexed onto a smaller-or-equal pool of kernel threads. Combines flexibility (create as many user threads as you like) with parallelism (multiple kernel threads). Complex to implement; used by some runtimes (e.g. Go's goroutine scheduler is a modern many-to-many-style design).



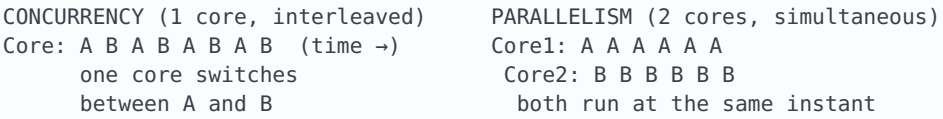
U = user thread, K = kernel thread. The mapping decides blocking behavior and parallelism.

Thread Control Block (TCB). Just as a process has a PCB, each thread has a lighter **TCB** holding its thread ID, register set, program counter, stack pointer, and scheduling state. Threads of one process share the PCB's memory and resource fields but each have their own TCB.

3.6 Concurrency vs Parallelism

These two words are used interchangeably in casual speech and must be distinguished precisely in an interview.

> ▣ **KEY CONCEPT**
>
> **Concurrency** = dealing with many things at once (structure): tasks are in progress over overlapping time periods, possibly interleaved on one core. **Parallelism** = doing many things at the exact same instant (execution): tasks literally run simultaneously on multiple cores. Concurrency is about how you structure work; parallelism is about how it executes. You can have concurrency on a single core (via interleaving); you need multiple cores for parallelism.



Concurrency interleaves; parallelism overlaps in real time. Parallel work is concurrent, but concurrent work need not be parallel.

Analogy. A single barista taking one order while steaming milk for another, alternating between them, is concurrent. Two baristas each making a full drink at the same time is parallel.

3.7 Multi-Core Systems

A modern CPU packs several **cores**, each a full processing unit able to run a thread independently. The OS scheduler spreads runnable threads across cores, achieving real parallelism. Some cores support **simultaneous multithreading** (Intel's Hyper-Threading), presenting two logical CPUs per physical core to better utilize execution units. To software, each logical CPU is just another place to run a thread — which is why one-to-one threading maps so naturally onto modern hardware.

3.8 Race Conditions and Critical Sections

The moment two threads share mutable data, correctness is at risk. This section introduces the problem; Part V solves it in full.

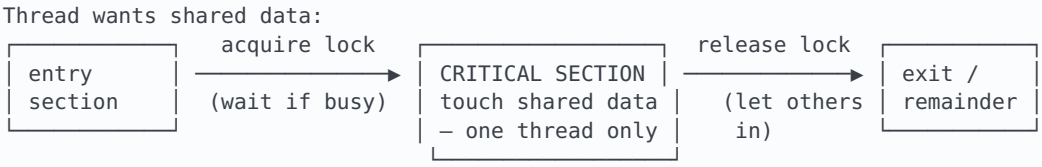
Race condition. A **race condition** occurs when the outcome of a program depends on the timing or interleaving of threads accessing shared data — and at least one is writing. The result is nondeterministic: it might work a thousand times and corrupt data on the thousand-and-first.

THE CLASSIC RACE: TWO THREADS INCREMENTING A SHARED COUNTER

```
// counter++ is NOT atomic. The CPU actually does three steps:
//   1) read  counter  into a register
//   2) add   1
//   3) write register back to counter

// If two threads interleave those steps on counter = 5:
//   T1 reads 5 ─┐                    T2 reads 5 ─┐
//   T1 adds → 6 │                    T2 adds → 6 │
//   T1 writes 6 ┘                    T2 writes 6 ┘
// Two increments happened, but counter is 6, not 7. One update was LOST.
```

Critical section. A **critical section** is a piece of code that accesses shared data and must not be executed by more than one thread at a time. The fix for a race is to enforce **mutual exclusion** over the critical section — only one thread inside at once — using a lock, mutex, or semaphore (Part V).



Guarding a critical section with a lock: enter one at a time, do the shared work, then release.

3.9 Thread Safety

Code is **thread-safe** if it behaves correctly when called from multiple threads at once, regardless of interleaving. You achieve it by (a) avoiding shared mutable state (use locals/immutable data), (b) protecting shared state with locks, or (c) using atomic operations. A function that keeps hidden shared state (like the classic `strtok`) is not thread-safe; reentrant variants (`strtok_r`) are provided precisely for threaded use.

A MINIMAL POSIX THREADS (PTHREADS) PROGRAM

```
#include <pthread.h>
#include <stdio.h>

void *worker(void *arg) {
    long id = (long)arg;
    printf("thread %ld running\n", id);
    return NULL;
}

int main(void) {
    pthread_t t[4];
    for (long i = 0; i < 4; i++)
        pthread_create(&t[i], NULL, worker, (void *)i);  // spawn
    for (int i = 0; i < 4; i++)
        pthread_join(t[i], NULL);                        // wait for each
    return 0;                 // compile with: gcc prog.c -lpthread
}
```

✓ Check Your Understanding — Part III

CONCEPTUAL

1. List exactly what threads of a process share and what each keeps private.
2. Give the precise difference between concurrency and parallelism, with an example of concurrency without parallelism.
3. Contrast user-level and kernel-level threads, especially blocking behavior.
4. Describe the three multithreading models and which one Linux uses.
5. Define a race condition and a critical section.

APPLICATION

1. Two threads each run `counter++` 1,000,000 times on a shared `counter` starting at 0, with no lock. What is the range of possible final values, and why?
2. You move a CPU-bound program from 4 threads to 40 threads on a 4-core machine and it gets slower. Explain.
3. Why can passing a pointer to a thread's local variable to another thread cause corruption?

INTERVIEW-LEVEL

1. "When would you choose multiple processes over multiple threads?"
2. "Is a single-core machine capable of concurrency? Of parallelism? Explain."

CAN YOU EXPLAIN THIS? (NO NOTES)

The shared-vs-private thread memory picture · concurrency vs parallelism · why `counter++` races · one-to-one vs many-to-one threading.

⚑ PRACTICAL TASK

Write a pthreads program where N threads each increment a shared global counter one million times with no lock. Print the final value. Run it several times — watch it come out different and wrong. Keep this program; in Part V you will fix it with a mutex and see it print the correct value every time.

Mastered when... you can draw the thread memory model, explain why shared mutable state races, and predict when adding threads helps versus hurts.

PART IV

CPU Scheduling

Many processes are Ready; only one CPU is free. Which one runs next? The scheduler answers that question thousands of times a second, and its policy decides whether your system feels snappy or sluggish, fair or starving.

In this part: Why scheduling · The metrics (turnaround, waiting, response, throughput) · Preemptive vs non-preemptive · FCFS · SJF · SRTF · Priority · Round Robin · Multilevel & multilevel-feedback queues · Starvation, aging, the convoy effect · Fully solved numerical problems

PART IV

CPU Scheduling

CPU scheduling is where operating-systems theory becomes arithmetic you can be tested on. This part teaches every classic algorithm the way an interviewer or examiner wants it: the intuition, the mechanism, a Gantt chart, and the full turnaround/waiting-time calculation worked to the last number.

4.1 Why CPU Scheduling? The Metrics

A process alternates CPU bursts with I/O bursts (Part II). While one process waits on I/O, the CPU would sit idle — a waste. The **scheduler** keeps the CPU busy by picking another Ready process to run. The **dispatcher** is the component that actually performs the context switch to hand the CPU over. The rules the scheduler follows are the **scheduling algorithm**, and we judge algorithms by these metrics:

Metric	Definition	Goal
Arrival Time (AT)	When the process enters the ready queue	— (given)
Burst Time (BT)	CPU time the process needs	— (given)
Completion Time (CT)	When the process finishes	— (computed)
Turnaround Time	TAT = CT − AT (total time in system)	Minimize
Waiting Time	WT = TAT − BT (time spent waiting in ready queue)	Minimize
Response Time	RT = (first time on CPU) − AT	Minimize
Throughput	Processes completed per unit time	Maximize
CPU Utilization	Fraction of time the CPU is busy	Maximize
Fairness	No process is unduly denied the CPU	Maximize

▣ **KEY CONCEPT — THE THREE FORMULAS THAT SOLVE EVERY PROBLEM**

Turnaround Time = Completion Time − Arrival Time

Waiting Time = Turnaround Time − Burst Time

Response Time = First-CPU Time − Arrival Time

Build the Gantt chart first, read off each Completion Time, then apply these. Every numerical in this part — and in your exam — is just these three lines applied carefully.

Two problems to watch for

Starvation

A process waits indefinitely because others keep being preferred (common with priority and SJF, where a low-priority or long job may never get picked).

Convoy effect

Short processes stuck waiting behind one long process (classic FCFS problem), tanking average waiting time — like a line of cars stuck behind one slow truck.

4.2 Preemptive vs Non-Preemptive

Aspect	Non-Preemptive	Preemptive
CPU taken away?	No — a process runs until it finishes or blocks	Yes — the OS can forcibly pause a running process
Triggered by	Process exits or requests I/O	Timer interrupt, higher-priority arrival
Responsiveness	Poorer (long job blocks others)	Better (can react to new/urgent work)
Overhead	Lower (fewer switches)	Higher (more context switches)
Examples	FCFS, SJF, non-preemptive Priority	SRTF, Round Robin, preemptive Priority

Modern general-purpose OSes are **preemptive** — essential for interactivity and to stop any one process from hogging the CPU.

4.3 FCFS — First-Come, First-Served

Intuition. A queue at a shop: serve whoever arrives first, to completion. **Working.** Non-preemptive; the ready queue is a simple FIFO. Simplest possible policy.

Worked example. Four processes:

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Order of service = order of arrival: P1, P2, P3, P4.



Boundaries: P1 0→5, P2 5→8, P3 8→16, P4 16→22.

Process	AT	BT	CT	TAT = CT−AT	WT = TAT−BT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average TAT = (5+7+14+19)/4 = **11.25** Average WT = (0+4+6+13)/4 = **5.75**

+ Simple, fair in arrival order, no starvation. **−** The **convoy effect**: put a long job first and every short job waits behind it, ballooning average waiting time. Non-preemptive, so bad for interactivity.

> ⚠ **COMMON MISTAKE**
>
> Forgetting idle time. If no process has arrived yet, the CPU sits idle and the timeline still advances. E.g. if P1 arrives at t=3 with nothing before it, the Gantt starts with idle [0→3], and P1's completion is 3+BT, not BT.

4.4 SJF — Shortest Job First (Non-Preemptive)

Intuition. Always run the Ready process with the smallest CPU burst next. Clearing the quick jobs first drags the average waiting time down. **SJF is provably optimal** for minimum average waiting time — among non-preemptive algorithms, no policy beats it.

Worked example (all arrive at t=0 for clarity): P1 BT=6, P2 BT=8, P3 BT=7, P4 BT=3. Pick shortest first: P4(3), P1(6), P3(7), P2(8).



Boundaries: P4 0→3, P1 3→9, P3 9→16, P2 16→24.

Process	BT	CT	TAT	WT
P4	3	3	3	0
P1	6	9	9	3
P3	7	16	16	9
P2	8	24	24	16

Average WT = (0+3+9+16)/4 = **7.0** (FCFS on the same set gives 10.25 — SJF wins)

+ Optimal average waiting time. **−** Requires knowing burst times in advance (impossible in general — the OS must estimate them, e.g. exponential averaging of past bursts); risks **starvation** of long jobs if short jobs keep arriving.

4.5 SRTF — Shortest Remaining Time First (Preemptive SJF)

Intuition. The preemptive version of SJF: whenever a new process arrives, compare its burst against the remaining time of the running process; if the newcomer is shorter, preempt. Decisions are re-made at every arrival.

Worked example: P1(AT=0,BT=8), P2(AT=1,BT=4), P3(AT=2,BT=9), P4(AT=3,BT=5).

- t=0: only P1 → run P1.
- t=1: P2(4) arrives; P1 remaining=7. 4<7 → preempt, run P2.
- t=2,3: P3(9), P4(5) arrive; P2 remaining (3, then 2) still smallest → keep P2.
- t=5: P2 done. Remaining P1=7, P3=9, P4=5 → run P4 (5).
- t=10: P4 done. Remaining P1=7, P3=9 → run P1 (7).
- t=17: P1 done. Run P3 (9) → done at 26.



Boundaries: P1 0→1, P2 1→5, P4 5→10, P1 10→17, P3 17→26.

Process	AT	BT	CT	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Average WT = (9+0+15+2)/4 = **6.5** Average TAT = (17+4+24+7)/4 = **13.0**

+ Lowest possible average waiting time of all these algorithms. **−** Heavy preemption overhead; still needs burst estimates; worst starvation risk for long jobs.

4.6 Priority Scheduling

Intuition. Each process has a priority number; the CPU always goes to the highest-priority Ready process. (Convention here: smaller number = higher priority.) Can be preemptive or non-preemptive. SJF is actually priority scheduling with priority = 1/burst.

Worked example (non-preemptive, all AT=0): P1(BT=4,pri=3), P2(BT=3,pri=1), P3(BT=1,pri=4), P4(BT=5,pri=2). Order by priority: P2, P4, P1, P3.



Boundaries: P2 0→3, P4 3→8, P1 8→12, P3 12→13.

WT: P2=0, P4=3, P1=8, P3=12 → Average WT = 23/4 = **5.75**

▣ **KEY CONCEPT — STARVATION & AGING**

Priority scheduling can **starve** low-priority processes forever if high-priority ones keep arriving. The standard cure is **aging**: gradually raise the priority of a process the longer it waits, so every process eventually becomes high-priority enough to run. (Legend: when MIT's IBM 7094 was shut down in 1973, a low-priority job submitted in 1967 was found still waiting.)

4.7 Round Robin (RR)

Intuition. FCFS with a stopwatch. Each process gets a fixed **time quantum** (time slice); if it doesn't finish, it's preempted and sent to the back of the ready queue. Designed for fairness and good response time — the heart of interactive time-sharing.

Worked example (quantum q=2, all AT=0): P1 BT=5, P2 BT=4, P3 BT=2, P4 BT=1.



Boundaries: P1 0→2, P2 2→4, P3 4→6, P4 6→7, P1 7→9, P2 9→11, P1 11→12.

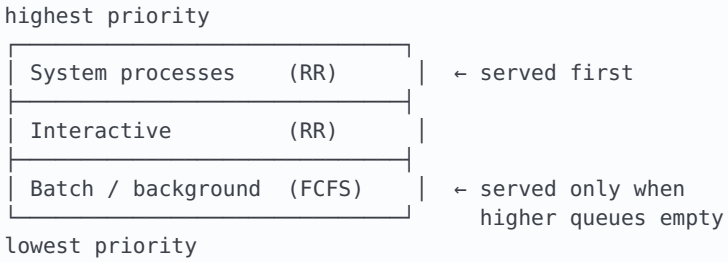
Process	BT	CT	TAT	WT	RT
P1	5	12	12	7	0
P2	4	11	11	7	2
P3	2	6	6	4	4
P4	1	7	7	6	6

Average WT = 24/4 = **6.0** Average TAT = 36/4 = **9.0** Average RT = 12/4 = **3.0**

> ⚑ **INTERVIEW TIP — CHOOSING THE QUANTUM**
>
> The time quantum is a knob. **Too large** → RR degenerates into FCFS (each job finishes in one slice), hurting response time. **Too small** → excessive context-switch overhead eats the CPU. Rule of thumb: the quantum should be large enough that ~80% of CPU bursts finish within it, typically 10–100 ms. RR optimizes response time and fairness, not average waiting time.

4.8 Multilevel Queue (MLQ)

Real systems have different classes of process (interactive, batch, system). MLQ splits the ready queue into several separate queues, each with its own scheduling algorithm, and processes are permanently assigned to one queue by type.



Multilevel queue: fixed classes, each with its own policy; higher queues preferred.

− Rigid — a process is stuck in its queue, and low queues can starve. That rigidity motivates the next algorithm.

4.9 Multilevel Feedback Queue (MLFQ)

The most sophisticated classic scheduler, and the closest to what real OSes use. Like MLQ, but processes can **move between queues** based on observed behavior:

- New processes enter the top (highest-priority, short-quantum) queue.
- A process that uses its whole quantum (CPU-bound) is **demoted** to a lower, longer-quantum queue.
- A process that yields early for I/O (interactive) stays high or is **promoted**.
- Periodic **aging** boosts long-waiting processes to prevent starvation.

▣ KEY CONCEPT

MLFQ learns each process's nature without being told its burst time: interactive jobs (short bursts) naturally float to the top for great responsiveness, while CPU hogs sink to the bottom where they run efficiently in long slices. It approximates SJF's benefits without needing to know burst times in advance. This is why it's the design basis for Windows, macOS, and older Linux schedulers.

4.10 Comparison & How to Choose

Algorithm	Preempt?	Optimizes	Starvation	Key weakness
FCFS	No	Simplicity	No	Convoy effect
SJF	No	Avg waiting time	Yes	Needs burst times
SRTF	Yes	Avg waiting time (best)	Yes	Overhead + estimates
Priority	Either	Importance	Yes	Starvation (use aging)
Round Robin	Yes	Response time, fairness	No	Quantum tuning
MLFQ	Yes	Balanced, adaptive	No (aging)	Complex to tune

Real-world: Linux's modern **CFS** (Completely Fair Scheduler) doesn't use fixed slices at all — it tracks each task's "virtual runtime" and always runs the one that has had the least CPU, approximating perfect fairness. The classic algorithms above remain the vocabulary for reasoning about and interviewing on scheduling.

✓ Check Your Understanding — Part IV

CONCEPTUAL

1. Write the formulas for turnaround, waiting, and response time.
2. What is the convoy effect and which algorithm suffers from it?
3. Why is SJF optimal for average waiting time, and why can't we use it directly?
4. Explain starvation and how aging fixes it.
5. How does MLFQ approximate SJF without knowing burst times?

APPLICATION (SOLVE FULLY — GANTT + TABLE)

1. P1(AT0,BT4), P2(AT1,BT3), P3(AT2,BT1), P4(AT3,BT2): compute average WT and TAT under **FCFS**, then under **SRTF**. Which is better?
2. Same four processes under **Round Robin, q=2**: give the Gantt chart and average response time.
3. Show that increasing RR's quantum toward infinity turns it into FCFS.

INTERVIEW-LEVEL

1. "Which scheduler would you pick for a real-time system vs. a batch server vs. a desktop, and why?"
2. "Design a scheduler that gives interactive apps good response time and batch jobs good throughput." (Hint: MLFQ.)

CAN YOU EXPLAIN THIS? (NO NOTES)

Build a Gantt chart and compute WT/TAT for any algorithm · convoy effect · the quantum trade-off in RR · why MLFQ adapts to process behavior.

⚑ **PRACTICAL TASK**

On Linux, run `nice -n 19 yes > /dev/null &` and `nice -n -5 yes > /dev/null &` (the second may need `sudo`), then watch them in `top`. Observe how the scheduler splits CPU by priority. Kill them with `kill %1 %2`. You've just driven the scheduler by hand (more in Part XIV).

Mastered when... given any process table you can produce the correct Gantt chart and average waiting/turnaround/response times for FCFS, SJF, SRTF, Priority, and Round Robin — and explain which metric each algorithm optimizes.

PART V

Process Synchronization

Concurrency without coordination is corruption. This part gives you the tools — mutexes, semaphores, monitors — and drills them on the three classic problems every OS course and systems interview returns to.

In this part: The critical-section problem · Mutual exclusion, progress, bounded waiting · Atomic operations · Locks & mutexes · Spinlocks · Busy waiting vs blocking · Semaphores (binary & counting) · Monitors & condition variables · Producer–Consumer · Readers–Writers · Dining Philosophers

PART V

Process Synchronization

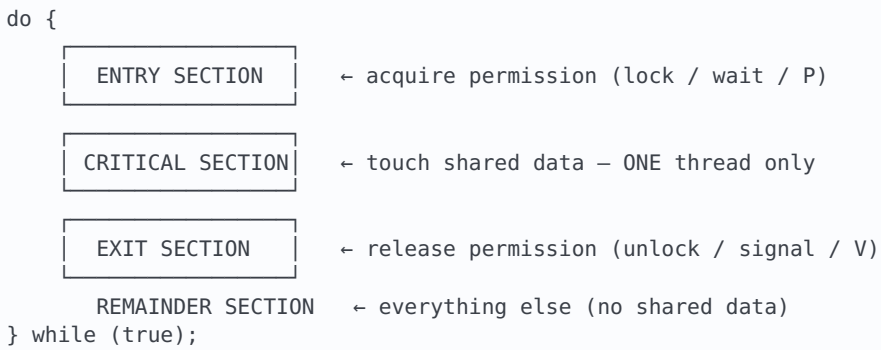
In Part III you saw `counter++` lose updates when two threads race. Here you learn to stop that — precisely, correctly, and without creating new problems like deadlock. These primitives are the vocabulary of every concurrent program you will ever write.

5.1 The Critical-Section Problem

Recall the race: shared data touched by concurrent threads, at least one writing, with a non-atomic operation in the middle. The region of code that touches the shared data is the **critical section**. The goal is to allow only one thread inside it at a time — **mutual exclusion** — without breaking two other guarantees.

> ▣ **KEY CONCEPT — THE THREE REQUIREMENTS OF A CORRECT SOLUTION**
>
> **1. Mutual Exclusion.** At most one thread executes in the critical section at a time.
> **2. Progress.** If no thread is in the critical section and some want in, the choice of who enters cannot be postponed indefinitely — a thread not interested must not block others.
> **3. Bounded Waiting.** There is a limit on how many times other threads enter before a waiting thread gets its turn — no starvation.

Every lock, semaphore, and monitor exists to satisfy these three at once. A "solution" that provides mutual exclusion but allows starvation (violates bounded waiting) is wrong.



The structure of any thread using a shared resource. The synchronization lives in the entry and exit sections.

5.2 Atomic Operations

What & why. An **atomic operation** executes as a single, indivisible step — it cannot be interrupted halfway, so no other thread can see a partial result. Atomicity is the bedrock: if `counter++` were atomic, there would be no race. Since it isn't, the hardware provides atomic primitives that locks are built from:

- **Test-and-Set** — atomically read a flag and set it to 1, returning the old value.
- **Compare-and-Swap (CAS)** — atomically: "if memory == expected, set it to new; return success." The workhorse of modern lock-free programming.
- **Fetch-and-Add** — atomically add to a memory location and return the old value.

A SPINLOCK BUILT FROM ATOMIC TEST-AND-SET (CONCEPT)

```
int lock = 0;                        // 0 = free, 1 = held

void acquire() {
    while (test_and_set(&lock) == 1)  // atomically: old=lock; lock=1; return old
        ;                             // spin: keep trying until we saw 0
}
void release() { lock = 0; }          // hand it back
```

5.3 Locks and Mutexes

A **mutex** (MUTual EXclusion lock) is the simplest synchronization object: it is either **locked** or **unlocked**, and it has an **owner**. A thread `lock()`s it before the critical section and `unlock()`s it after. Only the thread that locked it may unlock it. If another thread tries to lock a held mutex, it **blocks** (sleeps) until the owner releases.

FIXING THE PART III RACE WITH A PTHREAD MUTEX

```
#include <pthread.h>

long counter = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&m);       // ENTRY: only one thread past here
        counter++;                    // CRITICAL SECTION (now safe)
        pthread_mutex_unlock(&m);     // EXIT: let the next thread in
    }
    return NULL;
}
// Now the final counter is EXACTLY N * 1000000, every single run.
```

> ⚠ **COMMON MISTAKE**
>
> Locking too much or too little. Lock too little (leave a shared write unprotected) and the race persists. Lock too much (hold the mutex across slow I/O) and you serialize everything, killing concurrency. Hold locks for the shortest correct span — protect the shared data, nothing more.

5.4 Spinlocks: Busy Waiting vs Blocking

When a lock is held, a waiting thread can do one of two things:

Busy waiting (spinlock)
: Loop, repeatedly checking the lock ("spinning"), burning CPU while it waits. No context switch, so it's fast if the wait is very short.

Blocking (mutex)
: Give up the CPU and sleep; the OS wakes the thread when the lock frees. Wastes no CPU, but pays two context switches (sleep + wake).

	Spinlock (busy wait)	Mutex (blocking)
While waiting	Spins, consuming CPU	Sleeps, frees the CPU
Cost to wait	Wasted cycles	Two context switches
Best when	Very short critical sections, multi-core	Long or unpredictable waits
On 1 core	Terrible (spins while holder can't run)	Correct choice
Used in	Kernel, low-level, brief locks	Application code, general use

> ⚑ **INTERVIEW TIP**
>
> "Spinlock vs mutex?" A spinlock busy-waits; a mutex sleeps. Spinlocks win only when the critical section is shorter than the cost of two context switches and you have multiple cores (so the holder can run on another core while you spin). On a single core, spinning is pointless — the thread holding the lock can't make progress while you monopolize the CPU spinning. Real kernels use hybrid "adaptive" locks that spin briefly, then sleep.

5.5 Semaphores

What is it? A **semaphore** is an integer counter with two atomic operations, invented by Dijkstra. `wait()` (also **P**, or `down`) decrements it, blocking if it would go negative; `signal()` (also **V**, or `up`) increments it, waking a waiter. The counter tracks "how many units of a resource are available."

SEMAPHORE SEMANTICS (ATOMIC)

```
wait(S):   // P — acquire
    while (S <= 0) ;   // block until a unit is available
    S = S - 1;

signal(S): // V — release
    S = S + 1;         // wake a blocked waiter if any
```

Binary semaphore

Counter is only 0 or 1. Acts like a lock (mutual exclusion). But unlike a mutex, it has no owner — any thread may signal it.

Counting semaphore

Counter ranges over 0..N, representing N identical resource units (e.g. "5 database connections available"). Threads `wait()` to take one, `signal()` to return one.

▣ **KEY CONCEPT — TWO DISTINCT USES OF SEMAPHORES**

(1) Mutual exclusion: a binary semaphore initialized to 1 guards a critical section. **(2) Signaling / ordering:** a semaphore initialized to 0 makes one thread wait for another to reach a point (thread A does `signal`, thread B's `wait` unblocks). This second use — enforcing an order between threads — is something a plain mutex cannot do, and it's the key to producer–consumer.

5.6 Mutex vs Semaphore — the Comparisons

	Mutex	Binary Semaphore	Counting Semaphore
Value	Locked / unlocked	0 or 1	0 .. N
Ownership	**Yes** — locker must unlock	No — anyone can signal	No
Purpose	Mutual exclusion	Mutual exclusion / signaling	Manage N resources / signaling
Can signal from another thread?	No	Yes	Yes
Analogy	One key, you took it, you return it	One key on a hook, anyone can hang it	N keys on a hook

▣ **KEY CONCEPT — MUTEX VS SEMAPHORE IN ONE LINE**

A **mutex is a locking mechanism** (ownership; used for mutual exclusion). A **semaphore is a signaling mechanism** (a counter; used to coordinate/count). Every mutex can be built from a binary semaphore, but a mutex adds ownership. Use a **mutex** to protect data; use a **semaphore** to count resources or to make one thread wait for an event in another.

> ⚠ **COMMON MISTAKE**
>
> "A binary semaphore is just a mutex." Not quite — the difference is **ownership**. A mutex can only be unlocked by the thread that locked it; a binary semaphore can be signaled by any thread. That makes the semaphore usable for signaling between threads, but also makes it easier to misuse (e.g. accidentally signal twice). This exact distinction is a favorite trap question.

5.7 Monitors and Condition Variables

Semaphores are powerful but error-prone: forget a `signal`, or order two `wait`s wrongly, and you deadlock — with no compiler help. A **monitor** is a higher-level construct that bundles shared data and the operations on it, guaranteeing that only one thread is active inside the monitor at a time (automatic mutual exclusion). Languages provide them directly: Java's `synchronized`, C#'s `lock`.

Inside a monitor, **condition variables** let a thread wait for a condition without busy-waiting:

- `wait(cv)` — atomically release the monitor lock and sleep until signaled.
- `signal(cv)` — wake one thread waiting on `cv`.
- `broadcast(cv)` — wake all of them.

> ⚠ **COMMON MISTAKE**
>
> Checking a condition with `if` instead of `while` before `wait()`. After waking, a thread must recheck the condition in a loop, because (a) another thread may have changed it first (spurious/ stolen wakeups). Always: `while (!ready) cond_wait(&cv,&m);`

5.8 Classic Problem 1 — Producer–Consumer (Bounded Buffer)

Problem. A producer thread creates items and puts them in a fixed-size buffer; a consumer takes them out. **Why it's hard:** the producer must wait if the buffer is full; the consumer must wait if it's empty; and both mutate the buffer, so access must be mutually exclusive.

Naive (broken) approach: a single mutex around the buffer. It prevents corruption but doesn't handle full/empty — the producer would spin-fail on a full buffer, wasting CPU, and there's no clean way to wait for space. **Correct approach:** two counting semaphores to track space and items, plus a mutex for exclusion.

PRODUCER-CONSUMER WITH SEMAPHORES (THE CANONICAL SOLUTION)

```
sem_t empty;   // counts free slots; init to N (buffer size)
sem_t full;    // counts filled slots; init to 0
pthread_mutex_t mutex;  // protects the buffer; init unlocked
```

```
void producer() {
    while (1) {
        item = produce_item();
        sem_wait(&empty);          // wait for a free slot  (blocks if full)
        pthread_mutex_lock(&mutex);
        buffer_put(item);          // CRITICAL SECTION
        pthread_mutex_unlock(&mutex);
        sem_post(&full);           // announce one more item
    }
}
void consumer() {
    while (1) {
        sem_wait(&full);           // wait for an item  (blocks if empty)
        pthread_mutex_lock(&mutex);
        item = buffer_get();       // CRITICAL SECTION
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);          // announce one more free slot
        consume_item(item);
    }
}
```

⚠ **COMMON MISTAKE — DEADLY ORDERING BUG**

Never lock the mutex before the semaphore wait. If the producer does `lock(mutex)` then `wait(empty)` on a full buffer, it sleeps holding the mutex; the consumer can never lock the mutex to remove an item → **deadlock**. Order is always: `wait(counting semaphore)` first, then `lock(mutex)`.

5.9 Classic Problem 2 — Readers–Writers

Problem. A shared resource (e.g. a database) may be read by many threads simultaneously (reads don't conflict), but a writer needs exclusive access (writes conflict with everything). Maximize concurrency: allow concurrent readers, but a writer must be alone.

Correct approach (reader-preference): a mutex for the resource, a counter of active readers, and a mutex protecting that counter. The first reader in "locks out" writers; the last reader out releases them.

READERS-WRITERS (READER PREFERENCE)

```
int readers = 0;
pthread_mutex_t rc;          // protects the readers counter
sem_t rw;                    // exclusive access to the resource; init 1

void reader() {
    pthread_mutex_lock(&rc);
    if (++readers == 1) sem_wait(&rw); // FIRST reader blocks writers
    pthread_mutex_unlock(&rc);

    read_resource();                   // many readers here at once

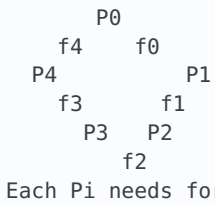
    pthread_mutex_lock(&rc);
```

```
    if (--readers == 0) sem_post(&rw); // LAST reader frees writers
    pthread_mutex_unlock(&rc);
}
void writer() {
    sem_wait(&rw);            // exclusive: wait for zero readers AND no writer
    write_resource();
    sem_post(&rw);
}
```

Trade-off / common mistake: this reader-preference version can **starve writers** — a steady stream of readers never lets `readers` hit 0. Production solutions use writer-preference or a fair queue to guarantee bounded waiting for writers.

5.10 Classic Problem 3 — Dining Philosophers

Problem. Five philosophers sit around a table; one fork between each pair (five forks). To eat, a philosopher needs both adjacent forks. This models processes competing for multiple shared resources — and it's the canonical way to produce (and study) **deadlock**.



Each Pi needs fork[i] (left) and fork[(i+1)%5] (right).

Five philosophers, five forks. Everyone grabbing their left fork at once = deadlock.

Naive (deadlocking) approach: everyone picks up their left fork, then their right. If all five grab their left fork simultaneously, each holds one and waits forever for the right — a perfect **circular wait** (Part VI). **Correct approaches** that break the circular wait:

- **Resource ordering:** number the forks; always pick up the lower-numbered fork first. One philosopher then reaches for forks in the opposite order, breaking the cycle. (Breaks circular wait.)
- **Asymmetry:** odd philosophers grab left-then-right, even ones right-then-left.
- **Limit diners:** allow at most 4 philosophers to sit at once (a counting semaphore init to 4), guaranteeing at least one can get both forks.
- **Waiter/monitor:** a central arbiter grants forks only when both are free (pick up both or neither).

DINING PHILOSOPHERS — DEADLOCK-FREE VIA RESOURCE ORDERING

```
void philosopher(int i) {
    int left = i, right = (i + 1) % 5;
    int first = (left < right) ? left : right;   // always lower-numbered fork first
    int second = (left < right) ? right : left;
    while (1) {
```

```
        think();
        sem_wait(&fork[first]);      // consistent global order → no cycle → no
deadlock
        sem_wait(&fork[second]);
        eat();
        sem_post(&fork[second]);
        sem_post(&fork[first]);
    }
}
```

⚑ INTERVIEW TIP

Dining Philosophers is really a deadlock question in disguise. Name the four Coffman conditions (Part VI) and show which your solution breaks: resource ordering and asymmetry break **circular wait**; "pick up both or neither" breaks **hold-and-wait**; the ≤4-diners rule ensures at least one philosopher can always proceed. Mentioning the Coffman connection is what earns the senior-level nod.

✓ Check Your Understanding — Part V

CONCEPTUAL

1. State the three requirements a correct critical-section solution must satisfy.
2. What is the difference between a mutex and a semaphore? Between a binary semaphore and a mutex?
3. When is a spinlock the right choice, and when is it disastrous?
4. Why must a condition-variable wait be inside a `while` loop, not an `if`?
5. How do the two counting semaphores in producer–consumer differ in purpose?

APPLICATION

1. In producer–consumer, show exactly how swapping `wait(empty)` and `lock(mutex)` causes deadlock.
2. Explain how the reader-preference readers–writers solution can starve a writer, and propose a fix.
3. For dining philosophers, prove that the resource-ordering solution cannot deadlock.

INTERVIEW-LEVEL

1. "Implement a thread-safe bounded queue. Which primitives, and why?"
2. "You have a lock-free counter requirement. How would compare-and-swap help?"

CAN YOU EXPLAIN THIS? (NO NOTES)

Mutex vs semaphore vs binary semaphore · the producer–consumer semaphore solution and its deadly ordering bug · how each dining-philosophers fix breaks a Coffman condition.

⚑ PRACTICAL TASK

Take your racing counter from Part III. Add a `pthread_mutex_t` around the increment and confirm the result is now always exactly correct. Then time both versions — measure how much the lock costs. You've now felt both correctness and the price of synchronization.

Mastered when... you can implement producer–consumer, readers–writers, and a deadlock-free dining philosophers from memory, and explain the purpose of every `wait`/`signal` and lock in each.

PART VI

Deadlocks

Give two threads two locks and let them grab in opposite orders, and your program freezes forever — no crash, no error, just silence. Deadlock is the dark side of the synchronization you just learned. This part shows how it arises and the four ways to handle it.

In this part: What deadlock is · The four Coffman conditions · Resource allocation graphs · Prevention · Avoidance & safe states · Banker's Algorithm (fully worked) · Detection & recovery · Deadlock vs starvation vs livelock

PART VI

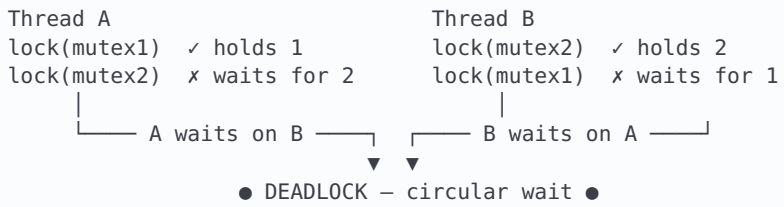
Deadlocks

A deadlock is a standstill: a set of processes each holding a resource the next one needs, in a circle, so none can ever proceed. Understanding the exact conditions that make it possible tells you precisely how to prevent it — and the Banker's Algorithm shows how to sidestep it entirely.

6.1 What Is a Deadlock?

What is it? A **deadlock** is a situation where a set of processes are each blocked, waiting for a resource that another process in the set is holding, so that none of them can ever make progress. The system is stuck — permanently, without intervention.

Simple intuition. A narrow bridge with one lane. Two cars enter from opposite ends and meet in the middle. Neither can pass, and neither will back up. Both wait forever. Each car holds "its half of the bridge" and needs "the other half" — held by the other car.



The textbook two-lock deadlock: two threads acquire the same two locks in opposite orders.

6.2 The Four Coffman Conditions

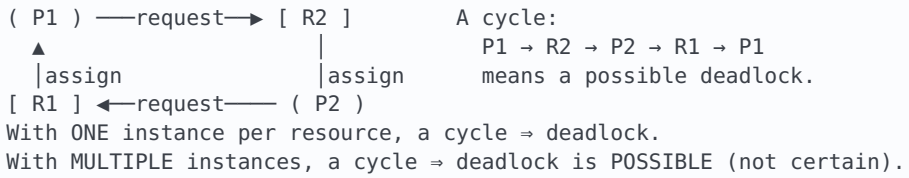
Deadlock is possible only if all four of these hold simultaneously. This is the single most important fact in the topic: break **any one** and deadlock cannot occur.

Condition	Meaning	Bridge analogy
1. Mutual Exclusion	At least one resource is non-shareable — only one process can hold it at a time.	The lane fits one car.
2. Hold and Wait	A process holds at least one resource while waiting to acquire others.	A car sits on its half while waiting for the other half.
3. No Preemption	Resources cannot be forcibly taken; they're released only voluntarily.	No one can make a car reverse.
4. Circular Wait	A closed chain of processes exists, each waiting for a resource held by the next.	Car A waits on B, B waits on A.

> ▣ **KEY CONCEPT**
>
> All four Coffman conditions are **necessary**; together they are **sufficient** (given a single instance of each resource). Every deadlock prevention strategy works by making one of the four impossible. Circular wait is usually the easiest to attack (via resource ordering), which is why "lock ordering" is the number-one practical deadlock defense.

6.3 Resource Allocation Graph (RAG)

A **resource-allocation graph** pictures who holds and who wants what. Processes are circles, resources are squares. An edge **R → P** (assignment) means the resource is allocated to the process; an edge **P → R** (request) means the process is waiting for it.



A cycle in the RAG signals deadlock (certain for single-instance resources, possible otherwise).

> ⚠ **COMMON MISTAKE**
>
> "A cycle in the resource-allocation graph always means deadlock." Only if every resource type has a single instance. With multiple instances of a resource, a cycle is necessary but not sufficient — another process outside the cycle might release an instance and break it.

6.4 Four Ways to Handle Deadlock

1. **Prevention** — structurally ensure one Coffman condition can never hold. (Conservative.)
2. **Avoidance** — allow the conditions but never enter a state that could lead to deadlock; the OS checks each request (Banker's Algorithm). (Needs future knowledge.)
3. **Detection & Recovery** — allow deadlocks, detect them with an algorithm, then recover (kill/rollback). (Reactive.)
4. **Ignore it** — the "ostrich algorithm": pretend deadlocks never happen. Used by most general-purpose OSes (Linux, Windows) because deadlocks are rare and prevention is costly; reboot if one occurs.

6.5 Deadlock Prevention

Attack each condition:

Break this	How	Cost
Mutual Exclusion	Make resources shareable (e.g. read-only files); use spooling	Not always possible (some resources are inherently exclusive)
Hold and Wait	Require a process to request all resources up front, or hold none while requesting	Low utilization; possible starvation
No Preemption	If a process must wait, forcibly release the resources it holds	Only works for state-saveable resources (CPU, memory — not a printer mid-print)
Circular Wait	Impose a global ordering on resources; always acquire in increasing order	Practical & popular — but you must define and honor the order everywhere

> ⚑ **INTERVIEW TIP**
>
> "How do you prevent deadlock in real code?" Answer: **lock ordering** — establish a fixed global order for acquiring locks and always take them in that order. This breaks circular wait and is what real codebases actually do. The two-lock example deadlocks precisely because A and B take `mutex1` / `mutex2` in opposite orders; force both to take `mutex1` first and it's impossible.

6.6 Deadlock Avoidance and Safe States

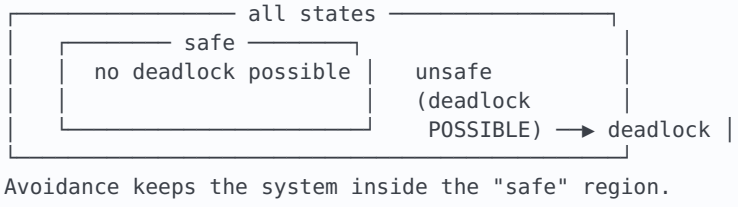
Avoidance is less restrictive than prevention: it permits the four conditions but, before granting any resource request, checks whether doing so keeps the system in a **safe state**.

Safe state

There exists at least one ordering of all processes (a **safe sequence**) in which each can obtain its maximum needed resources, run, and finish — even if every process demands its maximum. A safe state guarantees no deadlock.

Unsafe state

No such sequence is guaranteed to exist. Unsafe does not mean deadlock — it means deadlock is possible. Avoidance simply refuses to ever step into unsafe territory.



Avoidance keeps the system inside the "safe" region.

Safe ⊂ unsafe ⊃ deadlock. Avoidance never lets the system leave the safe region.

6.7 Banker's Algorithm (Worked Numerical)

The classic avoidance algorithm (Dijkstra). Like a banker who never lends so much that they can't satisfy every client's credit line, the OS grants a request only if the resulting state is safe.

Data structures (for n processes, m resource types): **Available[m]** (free units), **Max[n][m]** (each process's maximum claim), **Allocation[n][m]** (currently held), and **Need[n][m] = Max − Allocation**.

Example. Total resources: A=10, B=5, C=7. Snapshot of five processes:

Process	Allocation			Max			Need = Max−Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

Allocated totals: A=7, B=2, C=5, so **Available = (10,5,7) − (7,2,5) = (3, 3, 2)**.

Safety check. Start with Work = Available = (3,3,2) and all processes unfinished. Repeatedly find a process whose Need ≤ Work; "run" it and add its Allocation back to Work.

Step	Pick (Need ≤ Work?)	Need	Work before	Work after (+Alloc)
1	P1: (1,2,2) ≤ (3,3,2) ✓	1 2 2	3 3 2	5 3 2
2	P3: (0,1,1) ≤ (5,3,2) ✓	0 1 1	5 3 2	7 4 3
3	P0: (7,4,3) ≤ (7,4,3) ✓	7 4 3	7 4 3	7 5 3
4	P2: (6,0,0) ≤ (7,5,3) ✓	6 0 0	7 5 3	10 5 5
5	P4: (4,3,1) ≤ (10,5,5) ✓	4 3 1	10 5 5	10 5 7

All five processes finished ⇒ state is **SAFE**. Safe sequence = ⟨ P1, P3, P0, P2, P4 ⟩

A resource request. Suppose P1 now requests **(1, 0, 2)**. Banker's checks: (i) Request ≤ Need P1 (1,2,2)? ✓ (ii) Request ≤ Available (3,3,2)? ✓. Tentatively grant: Available → (2,3,0), Alloc(P1) → (3,0,2), Need(P1) → (0,2,0). Re-run the safety check — it still finds the sequence ⟨ P1, P3, P0, P2, P4 ⟩, so the state is safe and the request is **granted**. (Had it been unsafe, the request would be denied and P1 made to wait.)

> ⚑ **INTERVIEW TIP**
>
> The mechanical recipe: (1) compute **Need = Max − Allocation**; (2) **Available = Total − sum of Allocation**; (3) run the safety loop with `Work` and `Finish[]`. If you can find any order that finishes everyone, it's safe. For a request, check `Request ≤ Need` and `Request ≤ Available`, pretend-allocate, then re-run safety. Practice this until it's automatic — it's a guaranteed exam question (see Part XIX for more).

6.8 Deadlock Detection and Recovery

If you neither prevent nor avoid, you must **detect**. For single-instance resources, detection = finding a cycle in the (wait-for) graph. For multiple instances, an algorithm similar to the Banker's safety check runs periodically: if some processes can never finish, they're deadlocked.

Recovery options once a deadlock is found:

- **Process termination** — kill all deadlocked processes, or kill them one at a time (least-cost first) until the cycle breaks.
- **Resource preemption** — forcibly take a resource from one process and give it to another, **rolling back** the victim to a safe checkpoint. Must avoid starving the same victim repeatedly.

6.9 Deadlock vs Starvation vs Livelock

	Deadlock	Starvation	Livelock
What happens	Processes blocked forever in a circular wait	A process waits indefinitely while others proceed	Processes actively change state but make no progress
Are they running?	No — all blocked	Others run; the victim doesn't	Yes — busy, but uselessly
Cause	All four Coffman conditions	Unfair scheduling / resource policy	Overly polite retry logic (both keep yielding)
Analogy	Gridlocked intersection	Always skipped in line	Two people stepping aside in a hallway, repeatedly
Fix	Break a Coffman condition	Aging / fair scheduling	Add randomness / backoff

> ▣ **KEY CONCEPT**
>
> **Deadlock:** everyone stuck, no one moving. **Livelock:** everyone moving, no one progressing (they keep reacting to each other). **Starvation:** the system progresses, but one unlucky process never gets its turn. All three are "no progress" failures, but the mechanism differs — and so does the fix.

✓ Check Your Understanding — Part VI

CONCEPTUAL

1. Name the four Coffman conditions and give the bridge analogy for each.
2. Why must all four hold for deadlock, and what does that imply for prevention?
3. When does a cycle in a resource-allocation graph guarantee deadlock?
4. Define safe and unsafe states. Does unsafe mean deadlock?
5. Distinguish deadlock, livelock, and starvation.

APPLICATION

1. Given Available=(3,3,2) and the table in §6.7, is a request of (3,3,0) by P4 grantable? Show the safety check.
2. Rewrite the two-lock deadlock example so it cannot deadlock, and name the condition you broke.
3. A printer deadlock can't be fixed by preemption — why? Which resources can be preempted?

INTERVIEW-LEVEL

1. "Why do most real operating systems 'ignore' deadlock (ostrich algorithm)?"
2. "Walk me through the Banker's Algorithm and its main practical drawback." (Hint: needs max claims in advance.)

CAN YOU EXPLAIN THIS? (NO NOTES)

The four Coffman conditions and how to break each · safe vs unsafe vs deadlocked · the full Banker's safety check · deadlock vs livelock vs starvation.

⚑ **PRACTICAL TASK**

Write a two-thread C program that deadlocks: thread A locks `m1` then `m2`; thread B locks `m2` then `m1`; put a `sleep(1)` between the two locks so it deadlocks reliably. Run it — it hangs. Then fix it by making both threads lock in the same order, and watch it complete.

Mastered when... you can run the Banker's Algorithm on any snapshot to decide safety and grant/deny a request, and you can look at concurrent code and spot the lock-ordering bug that would deadlock it.

PART VII

Memory Management

RAM is a single shared array of bytes, yet every process believes it owns a private, contiguous memory from address zero. Memory management is the machinery — addresses, page tables, and the MMU — that makes that comfortable illusion real and safe.

In this part: Logical vs physical vs virtual addresses · Address binding · Contiguous allocation · Fixed & variable partitioning · Internal & external fragmentation · Compaction · Paging (pages, frames, page tables) · Address translation · TLB · Multi-level & inverted page tables · Segmentation

PART VII

Memory Management

Every process you met in Part II runs in its own address space. Here is how that address space is actually mapped onto physical RAM — the addresses, the page tables, and the hardware that translates one into the other billions of times per second.

7.1 Why Memory Management?

Main memory (RAM) is a large array of bytes, each with an address. The CPU fetches instructions and data from it directly. The OS's memory-management job is to (1) give each process the memory it needs, (2) keep processes from reading or writing each other's memory (protection), (3) let more processes run than would fit if each needed all its memory at once, and (4) do all this fast, since every instruction touches memory. The central trick is a level of **indirection** between the addresses a program uses and the physical locations they end up at.

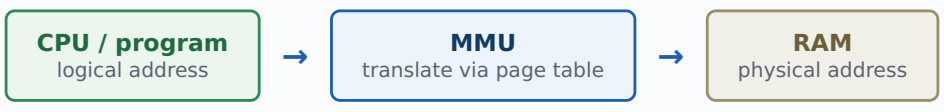
7.2 Logical, Physical, and Virtual Addresses

Logical (virtual) address
: The address a program generates and sees — e.g. the value of a pointer. It's relative to the process's own address space, starting conceptually at 0.

Physical address
: The real address of a byte in the RAM chips. What the memory hardware actually uses.

Virtual address
: Modern synonym for logical address in a system with virtual memory (Part VIII). The full set a process can name is its **virtual address space**.

The **Memory Management Unit (MMU)** — hardware in the CPU — translates every logical address to a physical address on the fly, using tables the OS sets up. The program never sees physical addresses; this is what enables both protection and virtual memory.



The MMU sits between the CPU and RAM, translating every memory reference.

Address Binding

"Binding" is fixing a program's addresses to real locations. It can happen at three times:

- **Compile time** — if you know where the program will load, generate absolute addresses. Rigid (must recompile to move).
- **Load time** — the compiler emits relocatable code; addresses are fixed when the program loads.
- **Execution time** — binding is deferred until run time and can change as the process moves; this needs hardware (the MMU) and is what modern systems use. It's what makes paging and swapping possible.

7.3 Contiguous Allocation

The oldest scheme: give each process a single contiguous block of physical memory.

Fixed (static) partitioning

Divide memory into a fixed number of partitions of preset size. Simple, but a process smaller than its partition wastes the rest (internal fragmentation), and a process larger than any partition can't run.

Variable (dynamic) partitioning

Partitions are created to fit each process exactly. No internal waste, but as processes come and go they leave holes, producing external fragmentation.

Placement strategies for variable partitioning — where to put a new process among the holes:

- **First fit** — first hole big enough. Fast.
- **Best fit** — smallest hole that fits. Minimizes leftover, but leaves many tiny unusable holes.
- **Worst fit** — largest hole. Leaves big usable leftovers (usually performs poorly in practice).

7.4 Fragmentation and Compaction

	Internal Fragmentation	External Fragmentation
What	Wasted space inside an allocated block (allocated but unused)	Free memory exists but is split into scattered small holes, none big enough
Cause	Fixed-size allocation larger than needed	Variable allocation over time
Example	Process needs 18 KB, gets a 20 KB partition → 2 KB wasted	3 free holes of 10 KB each, but a 25 KB process can't use them
Cure	Smaller/variable partitions; paging	Compaction; paging

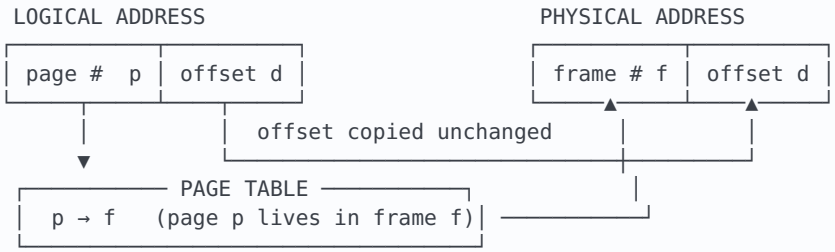
Compaction shuffles processes together to merge scattered holes into one big free block. It works only with execution-time binding (addresses can change), and it's expensive (lots of copying). The real cure for external fragmentation is **paging**, which drops the "contiguous" requirement entirely.

> ⚠ **COMMON MISTAKE**
>
> Swapping the two fragmentations. Mnemonic: **In**ternal = wasted space **in**side a block you were given. **Ex**ternal = usable space **ex**iled into scattered holes **ex**ternal to any allocation. Paging eliminates external fragmentation but introduces a little internal fragmentation (the last partial page).

7.5 Paging

The big idea. Stop requiring memory to be contiguous. Chop the process's virtual address space into fixed-size **pages** and physical memory into equal-size **frames** (a page and a frame are the same size, typically 4 KB). Any page can go in any free frame; a per-process **page table** records which frame holds each page. External fragmentation vanishes because any frame fits any page.

> ▣ **KEY CONCEPT — HOW AN ADDRESS IS SPLIT**
>
> A logical address is divided into two parts: a **page number** (high bits) and an **offset** (low bits) within the page. The page table maps the page number → a frame number; the offset is copied unchanged. With page size 2^k, the low **k** bits are the offset.
>
> physical address = frame_number × page_size + offset



Paged address translation: the page number indexes the page table to get a frame number; the offset passes through.

Worked example. Page size = 4 KB = 2^{12} bytes (so offset = 12 bits). Consider logical address **8196**:

- page number = 8196 ÷ 4096 = **2**; offset = 8196 mod 4096 = **4**.
- Suppose the page table says page 2 → frame **8**.
- physical address = 8 × 4096 + 4 = **32772**.

> ⚠ **COMMON MISTAKE**
>
> Thinking paging needs no translation cost. Naive paging doubles memory latency: one access to read the page table (in RAM) plus one to reach the actual data. That's exactly the problem the TLB solves.

7.6 The TLB (Translation Lookaside Buffer)

What & why. Since the page table lives in RAM, every memory reference would need an extra RAM access just to translate — halving performance. The **TLB** is a small, very fast associative cache inside the MMU that remembers the most recent page→frame translations.

- **TLB hit** — translation found in the TLB; frame number returned in ~1 cycle. Fast path.
- **TLB miss** — not cached; the MMU walks the page table in RAM (slow), then loads the result into the TLB for next time.

$$\text{Effective Access Time} = P_{\text{hit}}(\text{TLB} + \text{mem}) + P_{\text{miss}}(\text{TLB} + 2\times\text{mem})$$

Numerical: TLB access 10 ns, memory access 100 ns, hit ratio 90%. EAT = 0.9×(10+100) + 0.1×(10+200) = 99 + 21 = **120 ns** — versus 200 ns with no TLB. Real TLB hit rates exceed 99%, thanks to **locality** (Part VIII).

> ▣ **KEY CONCEPT**
>
> The TLB is why paging is practical. It exploits **locality of reference**: programs keep touching the same few pages, so a tiny cache of recent translations catches almost every access. On a context switch the TLB may be flushed (translations belong to the old process's page table) — one of the hidden costs of switching processes.

7.7 Multi-Level and Inverted Page Tables

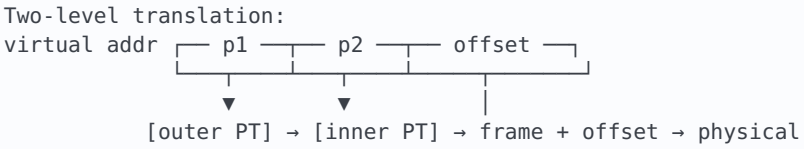
The problem: a single flat page table for a 32-bit address space with 4 KB pages needs $2^{20} \approx 1$ million entries per process — megabytes of table, mostly unused. Two solutions:

Multi-level (hierarchical) page tables

Page the page table itself. An outer table points to inner tables, which point to frames. Unused regions simply have no inner table, saving space. x86-64 uses **four levels**. Cost: a TLB miss now walks several levels (more memory accesses) — mitigated by the TLB.

Inverted page table

One table for the whole system, with one entry per physical frame (not per virtual page). Saves enormous space, but looking up "which frame holds page p of process X" requires searching — so it's paired with a hash table.



A hierarchical page table: the outer table indexes inner tables; empty regions cost nothing.

7.8 Segmentation

What. Instead of fixed-size pages, divide a program by its logical parts — **segments** like code, data, stack, heap — each a variable-length unit meaningful to the programmer. A logical address is <segment number, offset>; a **segment table** stores each segment's base and limit. This matches how programmers think and eases protection/sharing per segment (e.g. code read-only, shared between processes).

Downside: variable-size segments bring back **external fragmentation** (like variable partitioning). Pure segmentation is largely historical; modern systems use paging, sometimes **segmentation with paging** (segments divided into pages) to get both worlds.

7.9 Paging vs Segmentation

Aspect	Paging	Segmentation
Unit	Fixed-size page	Variable-size logical segment
Divided by	Hardware, arbitrary boundaries	Programmer's logical structure
Address	(page #, offset)	(segment #, offset)
Fragmentation	Internal (last page)	External (variable holes)
Table	Page table	Segment table (base + limit)
Visible to programmer?	No (transparent)	Yes (logical units)
Sharing/protection	Per page	Per segment (natural)

✓ Check Your Understanding — Part VII

CONCEPTUAL

1. Distinguish logical, physical, and virtual addresses, and say what the MMU does.
2. Contrast internal and external fragmentation, with the "in/ex" mnemonic.
3. How does paging eliminate external fragmentation? What does it introduce instead?
4. Why does naive paging halve memory performance, and how does the TLB fix it?
5. Why are multi-level page tables more space-efficient than a flat one?

APPLICATION

1. Page size 1 KB. Translate logical address 3500 given page 3 → frame 7. (Find page #, offset, physical address.)
2. TLB access 20 ns, memory 100 ns, hit ratio 80%. Compute the effective access time.
3. A 32-bit address space with 4 KB pages: how many entries in a single-level page table? Why is that a problem?

INTERVIEW-LEVEL

1. "Walk me through translating a virtual address to physical, including a TLB miss."
2. "Why do modern CPUs use paging instead of segmentation?"

CAN YOU EXPLAIN THIS? (NO NOTES)

Logical→physical translation via page table · page/offset splitting math · the role and payoff of the TLB · internal vs external fragmentation.

⚑ PRACTICAL TASK

On Linux, run `getconf PAGE_SIZE` (usually 4096). Then `cat /proc/self/maps` to see your shell's virtual memory regions — code, heap, stack, shared libraries — each an address range with permissions. You're looking at a real process's virtual address space.

Mastered when... given a page size and page table you can translate any logical address to physical by hand, compute effective access time with a TLB, and explain why paging beats contiguous allocation.

PART VIII

Virtual Memory

Your programs routinely use more memory than your machine physically has, and it just works. Virtual memory is the illusion — backed by the disk, demand paging, and page replacement — that lets a 16 GB machine run 40 GB of programs.

In this part: What & why virtual memory · Demand paging · Page faults & their handling · Page replacement: FIFO, LRU, Optimal, Clock · Belady's anomaly · Thrashing · Working set & locality · Copy-on-write · Memory-mapped files · Solved page-replacement numericals

PART VIII

Virtual Memory

Paging (Part VII) let a process's memory be scattered across frames. Virtual memory takes the next leap: a page need not be in RAM at all. Keep only the pages a process is actively using in memory and park the rest on disk, and suddenly the machine can run far more than fits.

8.1 What Is Virtual Memory, and Why?

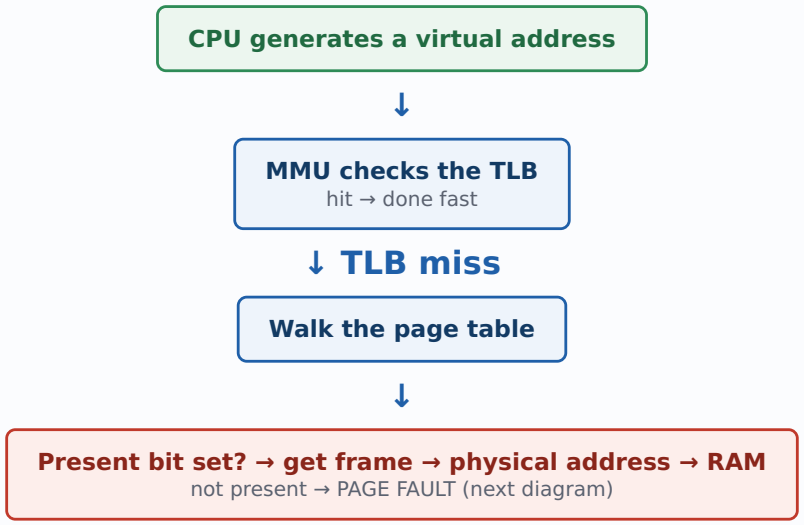
What is it? **Virtual memory** is a technique that lets a process execute even when only part of it is in physical RAM; the rest lives on disk (in the swap area or the program file) and is brought in on demand. Each process gets a large virtual address space independent of how much RAM exists.

Why? (1) Programs can be larger than physical memory. (2) More processes fit in RAM at once (only their active pages are resident), raising CPU utilization and multiprogramming. (3) Less I/O to load a program — start running before it's fully loaded. (4) It cleanly enables per-process isolation and features like copy-on-write and memory-mapped files.

> ▣ **KEY CONCEPT**
>
> Virtual memory works because programs exhibit **locality of reference**: at any moment a process touches only a small subset of its pages (its working set). Keep that hot subset in RAM and almost every access is a fast hit; the cold majority can sit on disk unnoticed.

8.2 Demand Paging

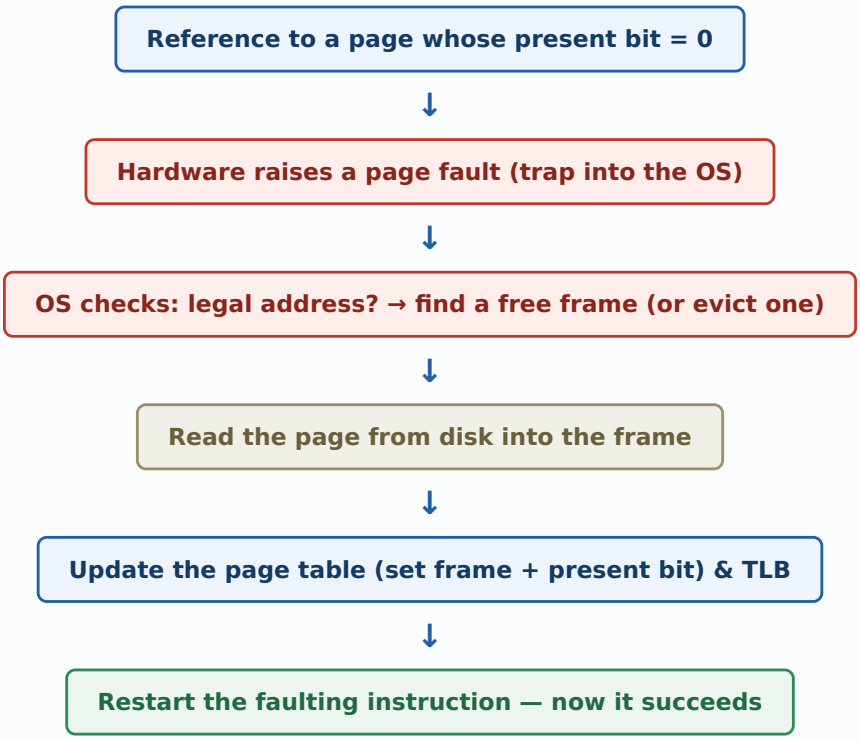
Demand paging is the usual implementation: don't load a page until it's actually referenced ("load on demand"). Each page-table entry has a **valid/invalid (present) bit**. When a page is in RAM the bit says "valid" and translation proceeds normally. When the page is on disk the bit says "invalid," and touching it triggers a **page fault**.



Address translation with virtual memory: TLB → page table → physical address, or a page fault.

8.3 Page Faults and Their Handling

A **page fault** is the exception raised when a process references a page that is not currently in RAM. It is not an error — it's the normal signal that the OS must fetch the page. Handling it:



Page-fault handling. The instruction that faulted is re-executed after the page is loaded.

Cost. A page fault is enormously expensive — a disk read is ~10^5–10^6× slower than a RAM access. So the **effective access time** is dominated by the fault rate:

$$\text{EAT} = (1 - p) \times \text{memory_access} + p \times \text{page_fault_time}$$

With memory access 100 ns and a page fault costing 8 ms (8,000,000 ns), even a fault probability of p = 0.001 gives EAT ≈ 100 + 0.001×8,000,000 = **8100 ns** — 81× slower. This is why keeping the fault rate tiny (via good replacement and enough RAM) is everything.

8.4 Page Replacement Algorithms

When a page fault occurs and no frame is free, the OS must **evict** a resident page to make room. Which one? A good choice minimizes future faults. We measure algorithms by counting page faults on a **reference string** (the sequence of page numbers a process touches).

Shared example. Reference string `1 2 3 4 1 2 5 1 2 3 4 5`, 3 frames.

FIFO — First-In, First-Out

Evict the page that has been in memory longest (oldest arrival). Simple; uses a queue. But "oldest" isn't "least useful," so it can evict a hot page.

Ref	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	4	4	4	5	5	5	5	5	5
Frame 2	·	2	2	2	1	1	1	1	1	3	3	3
Frame 3	·	·	3	3	3	2	2	2	2	2	4	4
Fault?	F	F	F	F	F	F	F	·	·	F	F	·

FIFO: **9 page faults** (3 hits)

Optimal (OPT / MIN)

Evict the page that will not be used for the longest time in the future. Provably the best possible — the fewest faults of any algorithm. It's unimplementable (needs the future), but it's the gold standard we compare against. On the same string, OPT gives **7 faults**.

LRU — Least Recently Used

Evict the page not used for the longest time in the past. A practical approximation of Optimal, banking on locality (the recent past predicts the near future). Costly to implement exactly (needs a timestamp or ordering per access). On the same string, LRU gives **10 faults** here.

Clock (Second-Chance)

A cheap LRU approximation used in real kernels. Frames sit in a circular list; each has a **reference bit** set on use. A "clock hand" sweeps: if the bit is 1, clear it and give the page a "second chance" (skip it); if 0, evict it. It approximates LRU with just one bit and no per-access bookkeeping.

> ▣ **KEY CONCEPT — BELADY'S ANOMALY**
>
> Intuitively, more frames should mean fewer faults. For **FIFO**, that can fail: on the string 1 2 3 4 1 2 5 1 2 3 4 5, FIFO gives **9 faults with 3 frames** but **10 faults with 4 frames** — more memory, more faults! This is **Belady's anomaly**. It afflicts FIFO but never **stack algorithms** like LRU and Optimal (which guarantee the set of pages with k frames is always a subset of the set with k+1 frames).

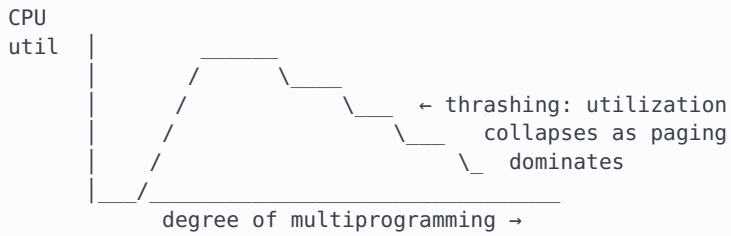
Algorithm (3 frames)	Page faults	Note
Optimal	7	Theoretical best
FIFO	9	Suffers Belady's anomaly
LRU	10	Approximates Optimal; no anomaly
FIFO (4 frames)	10	More frames, more faults!

8.5 Thrashing, Working Set, and Locality

Locality of reference is the empirical fact that processes access a small, slowly-changing set of pages at a time (loops reuse the same code; arrays are walked in order). Two flavors: **temporal** (a byte used now is likely used again soon) and **spatial** (nearby bytes are likely used soon). Locality is why caches and virtual memory work at all.

The **working set** of a process is the set of pages it has referenced in the recent time window — essentially its current locality. If RAM can hold every process's working set, fault rates stay low. If it can't…

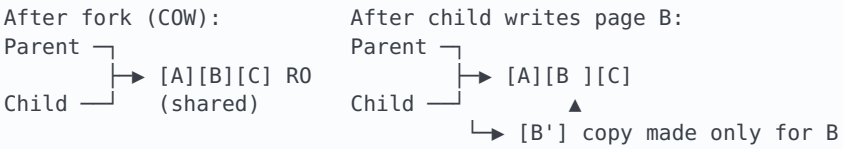
> ▣ **KEY CONCEPT — THRASHING**
>
> **Thrashing** is when a system spends more time paging (swapping pages in and out) than executing. It happens when the combined working sets exceed physical memory: every process constantly faults, evicting a page another process needs immediately, which faults again. CPU utilization collapses even though the CPU looks "busy" — busy doing I/O for paging. The fixes: reduce the multiprogramming level (run fewer processes), add RAM, or use the **working-set model**/page-fault-frequency control to give each process enough frames.



Add processes and CPU utilization rises — until working sets no longer fit and thrashing crashes it.

8.6 Copy-on-Write (COW)

What & why. Recall `fork()` (Part II) logically copies the parent's entire address space — wasteful, since the child often calls `exec()` immediately and discards it. **Copy-on-write** avoids the copy: parent and child share the same physical pages, marked read-only. As long as both only read, they share. The instant either writes a shared page, the MMU traps, and the OS makes a private copy of just that one page for the writer. Copying is deferred until (and only for pages) actually modified.



COW shares pages until a write forces a per-page copy — making *`fork()`* fast and lazy.

COW makes `fork()` nearly free and is used well beyond it — e.g. loading the same shared library into many processes without duplicating it in RAM.

8.7 Memory-Mapped Files

What. Instead of `read()`/`write()` system calls, a process can `mmap()` a file directly into its virtual address space, so the file appears as an array in memory. Accessing address `ptr[i]` reads byte `i` of the file; the OS demand-pages file contents in as you touch them and writes dirty pages back. **Why:** it avoids explicit I/O calls and extra copying, and it lets multiple processes share the same file pages in memory (the basis of shared memory IPC — Part XI, and of how executables/libraries load).

✓ Check Your Understanding — Part VIII

CONCEPTUAL

1. What is virtual memory and which property of programs makes it effective?
2. Describe the full sequence of events in handling a page fault.
3. Why is a page fault so much more expensive than a normal memory access?
4. What is Belady's anomaly, and which algorithms are immune to it?
5. Define thrashing and explain the working-set model's cure.

APPLICATION (SOLVE FULLY)

1. Reference string `7 0 1 2 0 3 0 4 2 3 0 3 2`, 3 frames: count page faults under FIFO, LRU, and Optimal.
2. Memory access 100 ns, page-fault service 5 ms, fault rate 0.0005. Compute the effective access time.
3. Explain, step by step, how copy-on-write handles `fork()` followed by the child writing one global variable.

INTERVIEW-LEVEL

1. "What happens, in hardware and software, on a page fault?"
2. "Your server's throughput suddenly collapses and disk I/O spikes. What's likely happening and how do you fix it?" (Thrashing.)

CAN YOU EXPLAIN THIS? (NO NOTES)

The page-fault handling flow · FIFO vs LRU vs Optimal (and Belady) · thrashing and the working set · how copy-on-write makes `fork()` cheap.

⚑ **PRACTICAL TASK**

On Linux run `vmstat 1` and watch the `si` / `so` (swap-in/swap-out) columns while opening many heavy apps. Nonzero, sustained values there are thrashing in action. Also try `/usr/bin/time -v ./yourprog` and read "Major (requiring I/O) page faults."

Mastered when... you can trace any reference string through FIFO/LRU/Optimal counting faults, explain and reproduce Belady's anomaly, and diagnose thrashing from system metrics.